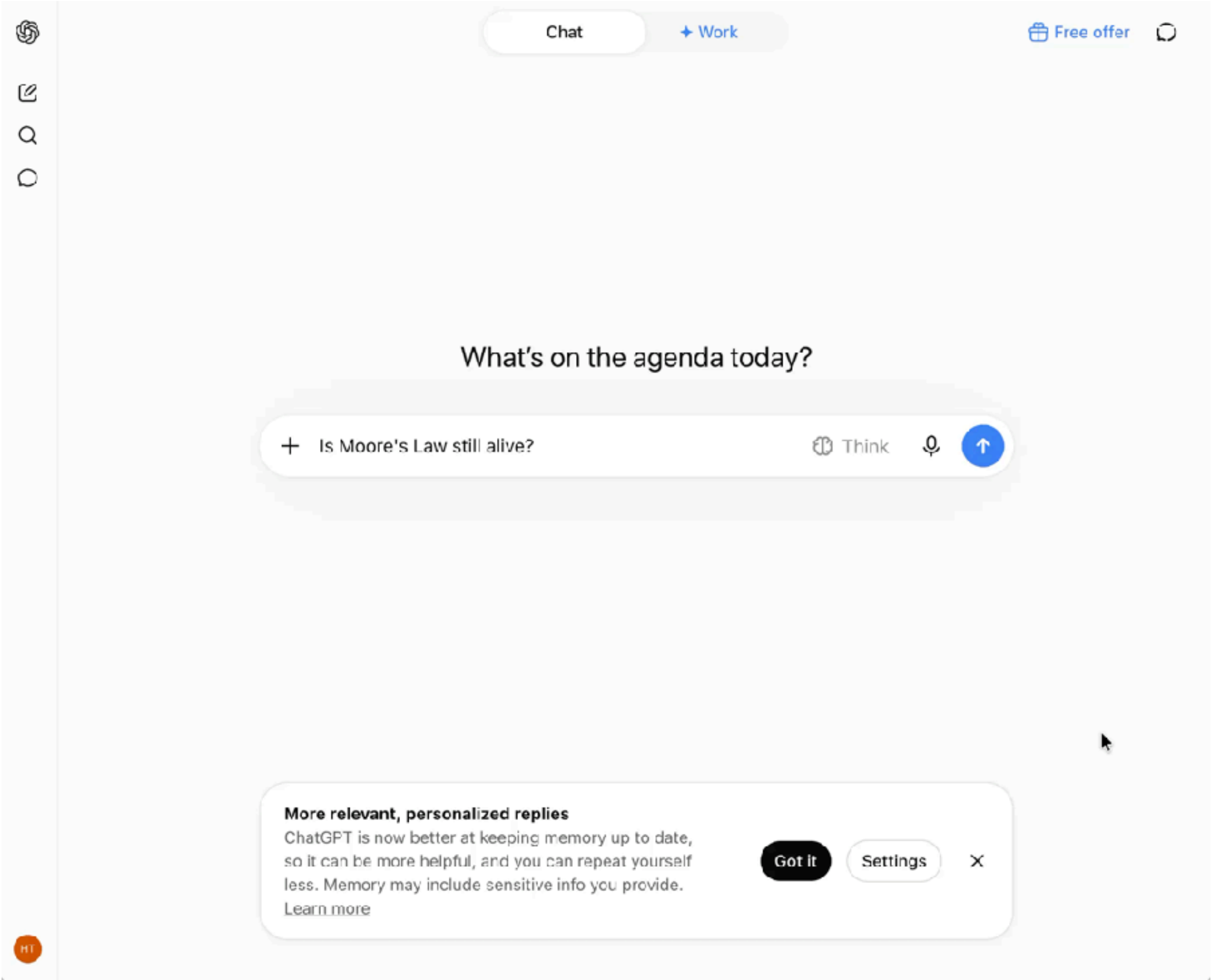
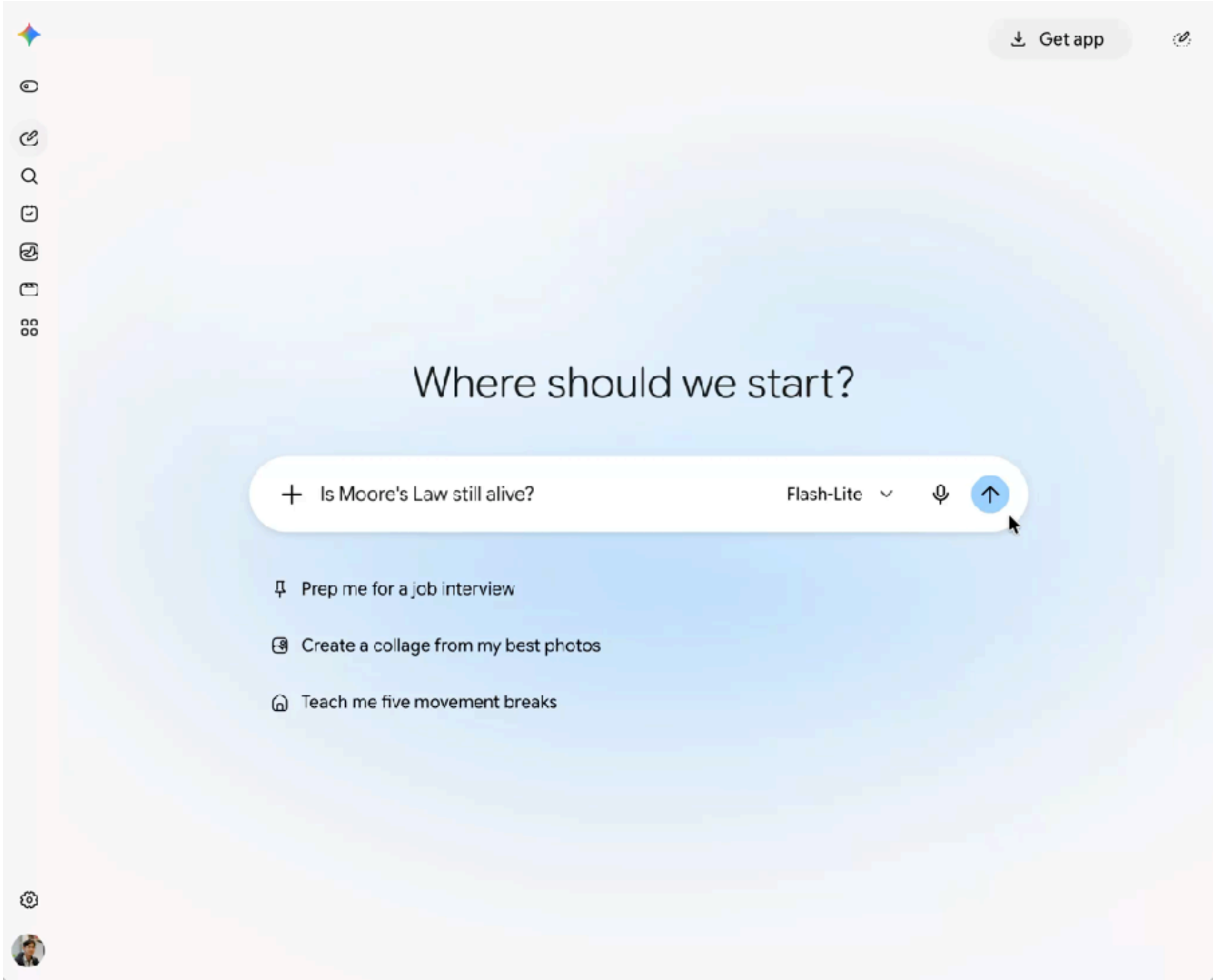


Performance

Hung-Wei Tseng

Moore's law — which side's performance is better?

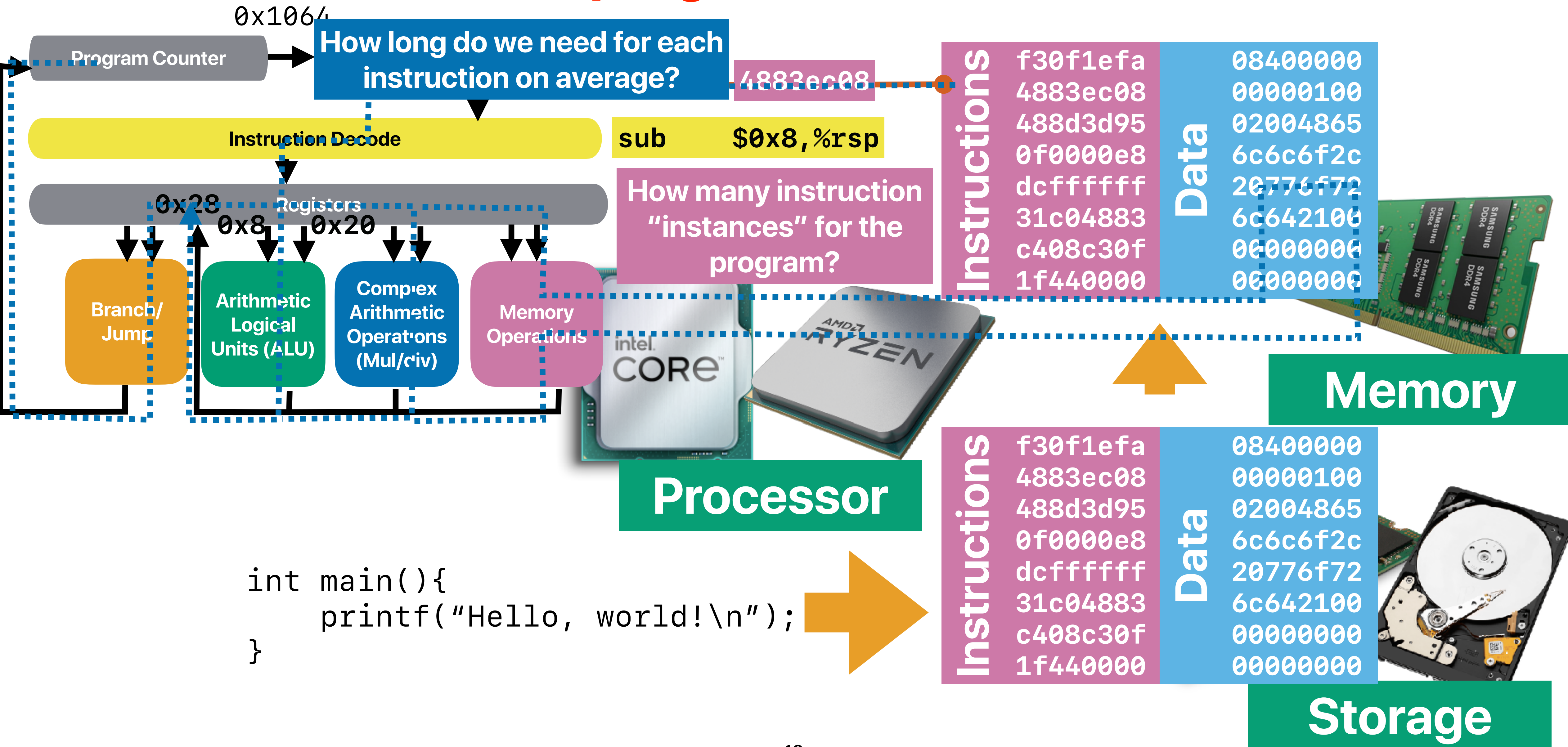


What do you compare matters!

Outline

- Definition of "Performance"
- The classical CPU performance equation
- What about metrics on throughputs? (e.g., FLOPS)
- What affects performance
 - Programming languages
 - Programmers

Execution time of a program in the von Neumann model



Definition of performance in Computer Architecture

$$\textit{Performance} = \frac{1}{\textit{Execution Time}}$$

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric —
Instruction count, cycles per instruction, cycle time define the
latency of execution on CPUs

What about FLOPS?

On the DeepSeek-V3 671B model, using 256 GPUs, Megatron Core reaches **1,648 TFLOPs per GPU** on GB300 NVL72, compared to **606 TFLOPs per GPU** on earlier GB200 NVL72 results, ~3x higher delivered throughput per GPU in a single generation.

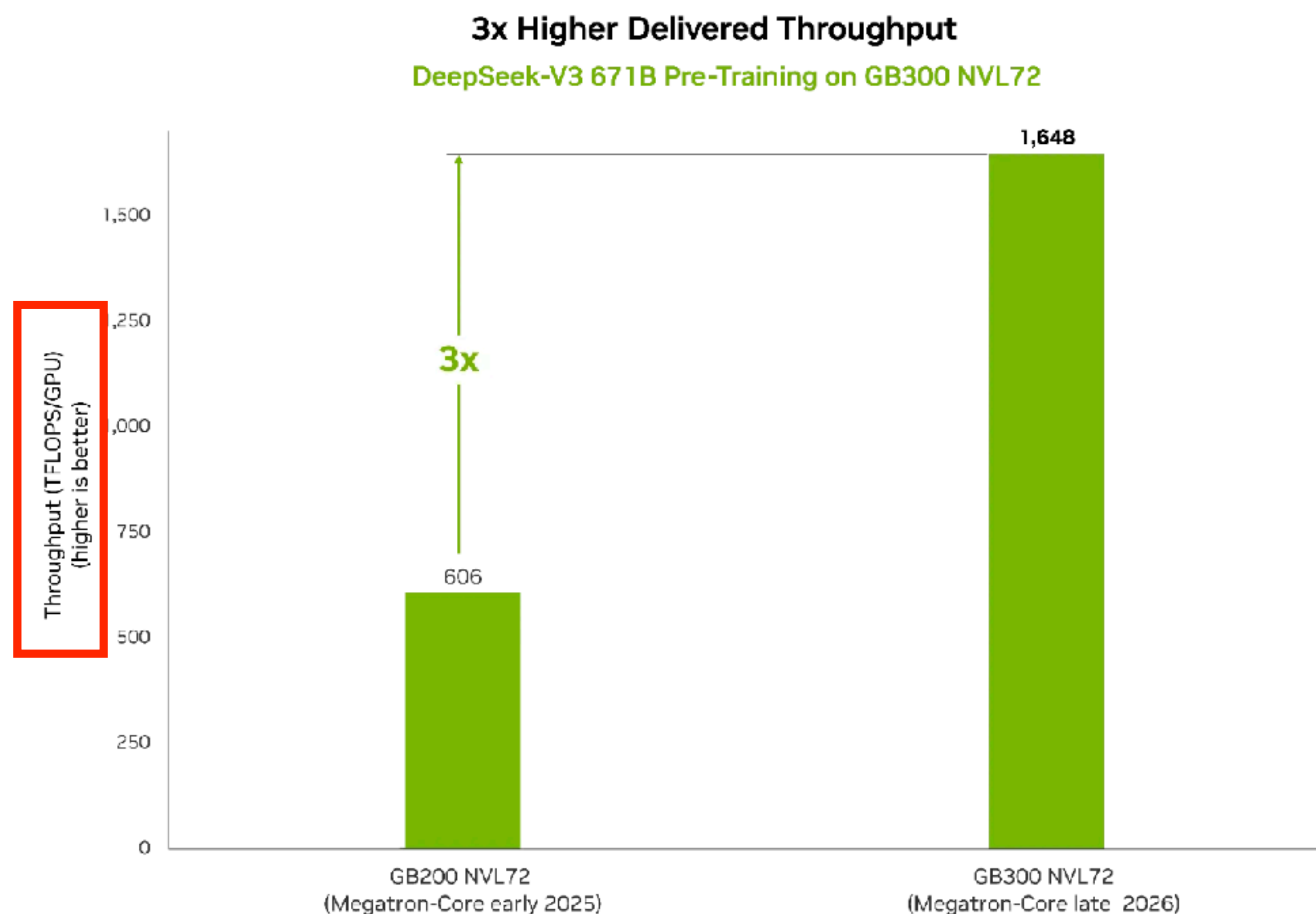


Figure 3. 3x higher delivered performance GB200 NVL72 (early software version) vs GB300 NVL72 (latest software version)

TFLOPS (Tera FLoating-point Operations Per Second)

$$TFLOPS = \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Exection Time}}$$

Let's measure the TFLOPS of matrix multiplications

```
for(i = 0; i < M; i++) {  
  for(j = 0; j < N; j++) {  
    for(k = 0; k < K; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

Floating point operations per second (FLOP"S"):

Floating point operations (FLOP"s"):

$$M \times N \times K \times 2$$

Given $M = N = K = 2048$

$$2^{3 \times 11} \times 2 = 2^{34} \quad \textbf{FLOPs in total}$$

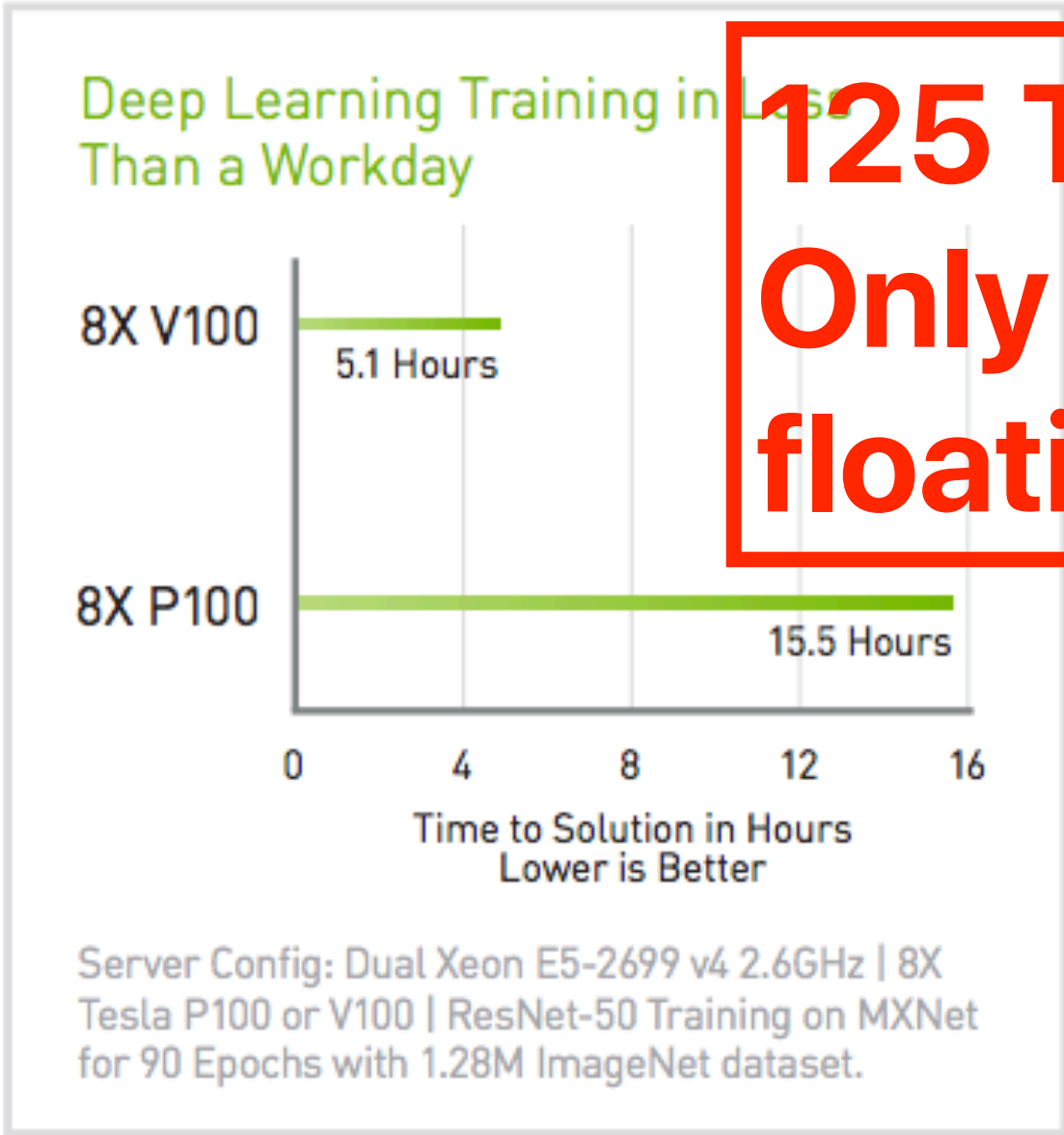
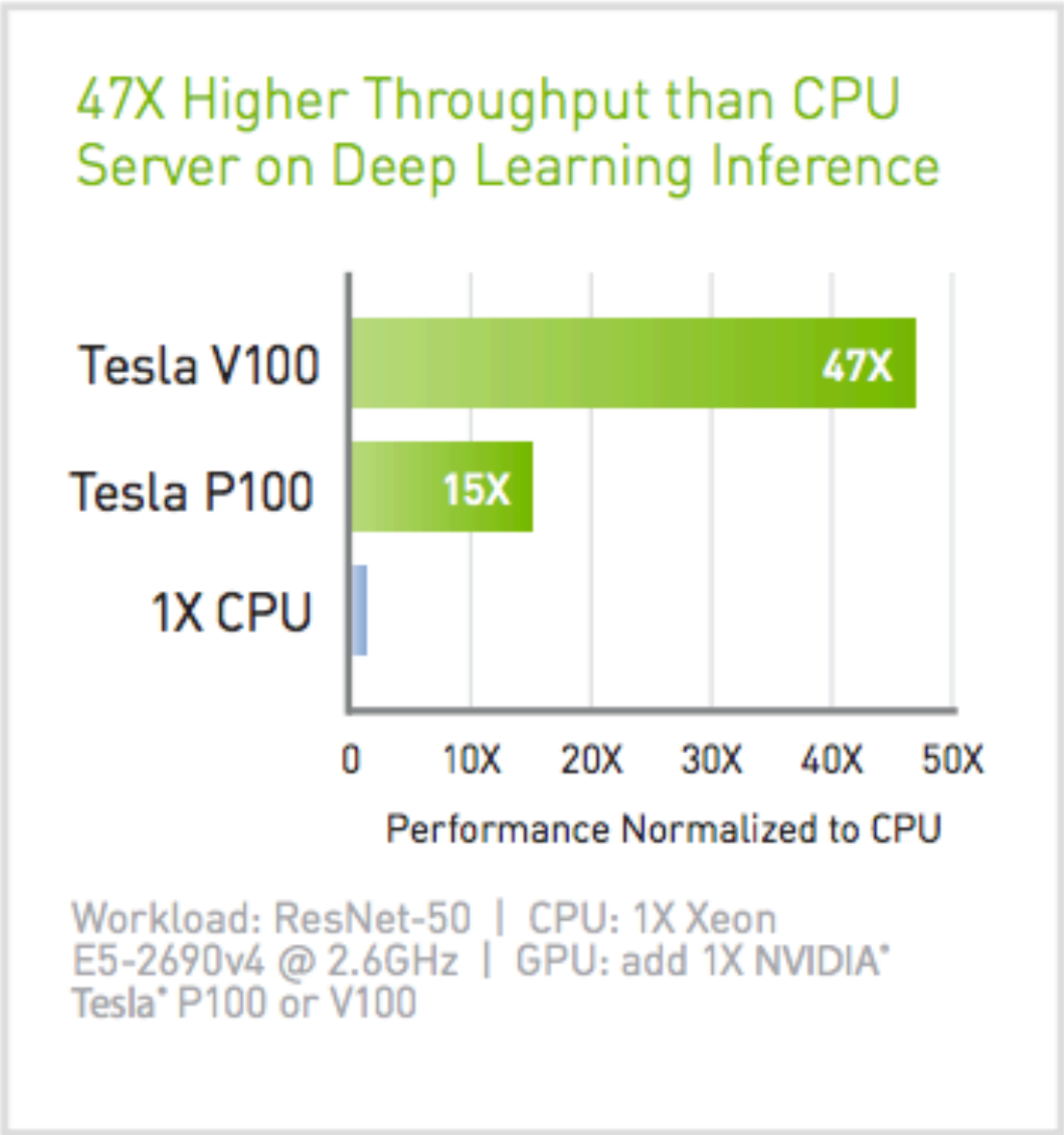
$$TFLOPS = \frac{2^{34}}{ET_{seconds} \times 2^{12}}$$

TFLOPS (Tera FLoating-point Operations Per Second)

$$TFLOPS = \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Exection Time}}$$

The Most Advanced Data Center GPU Ever Built.

NVIDIA® Tesla® V100 is the world's most advanced data center GPU ever built to accelerate AI, HPC, and graphics. Powered by NVIDIA Volta, the latest GPU architecture, Tesla V100 offers the performance of up to 100 CPUs in a single GPU—enabling data scientists, researchers, and engineers to tackle challenges that were once thought impossible.

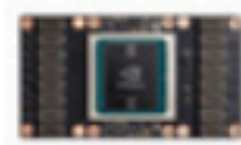


125 TFLOPS
Only @ 16-bit
floating point

SPECIFICATIONS



Tesla V100 PCIe



Tesla V100 SXM2

GPU Architecture	NVIDIA Volta	
NVIDIA Tensor Cores	640	
NVIDIA CUDA® Cores	5,120	
Double-Precision Performance	7 TFLOPS	7.8 TFLOPS
Single-Precision Performance	14 TFLOPS	15.7 TFLOPS
Tensor Performance	112 TFLOPS	125 TFLOPS
GPU Memory	32GB /16GB HBM2	
Memory Bandwidth	900GB/sec	
ECC	Yes	
Interconnect Bandwidth	32GB/sec	300GB/sec
System Interface	PCIe Gen3	NVIDIA NVLink
Form Factor	PCIe Full Height/Length	SXM2
Max Power	300W	300W

Is TFLOPS (Tera Floating-point Operations Per Second) a good metric?

$$\begin{aligned} TFLOPS &= \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Execution Time}} \\ &= \frac{IC \times \% \text{ of floating point instructions} \times 10^{-12}}{IC \times CPI \times CT} \\ &= \frac{\% \text{ of floating point instructions} \times 10^{-12}}{CPI \times CT} \end{aligned} \quad \text{IC is gone!}$$

- If we have more iterations? Larger datasets? — potentially changes the IC
- What if the hardware trade (cheat) performance with accuracy?
- Cannot compare different ISA/compiler
 - What if the compiler can generate code with fewer instructions?
 - What if new architecture has more IC but also lower CPI?
- If floating point operations are not critical in the target application?

AI training vs. inference

Much of the news coverage recently has been on LLMs, their development and their training – and the high cost and energy consumption required to do so. A recent study¹ estimated that Chat GPT3, comprised of 175 billion parameters, needed 1,287 MWh to train, and also emitted 552 tons of CO2. This is roughly the equivalent to driving a car 1.3 million miles – but in one hour!

But those huge training workloads running in data centers represent just 15 percent of AI workloads today, according to Omdia's Data Center Compute Intelligence Service.²

The rest – 85 percent of all data center AI workloads – lies in AI inference, and that's not accounting for inference that's happening outside the data center on the edge. AI workloads like inference will remain a key workload in the cloud, but an increasing number will move to the edge as more efficient models continue to evolve. Inference, which is the process of using a trained model that has been deployed into a production environment to make predictions on new real-world

The rest – 85 percent of all data center AI workloads – lies in AI inference, and that's not accounting for inference that's happening outside the data center on the edge. AI workloads like inference will remain a key

Inferences per second

$$\begin{aligned}\frac{\textit{Inferences}}{\textit{Second}} &= \frac{\textit{Inferences}}{\textit{Operation}} \times \frac{\textit{Operations}}{\textit{Second}} \\ &= \frac{\textit{Inferences}}{\textit{Operation}} \times \left[\frac{\textit{operations}}{\textit{cycle}} \times \frac{\textit{cycles}}{\textit{second}} \times \#_of_PEs \times \textit{Utilization_of_PEs} \right]\end{aligned}$$

Latency v.s. Bandwidth/Throughput

- Latency — the amount of time to finish an operation
 - End-to-end execution time of "something"
 - Access time
 - Response time
- Throughput — the amount of work can be done within a given period of time (typically "something" per "timeframe" or the other way around)
 - Bandwidth (MB/Sec, GB/Sec, Mbps, Gbps)
 - IOPs (I/O operations per second)
 - FLOPs (Floating-point operations per second)
 - IPS (Inferences per second)
 - FPS (Frames per second)

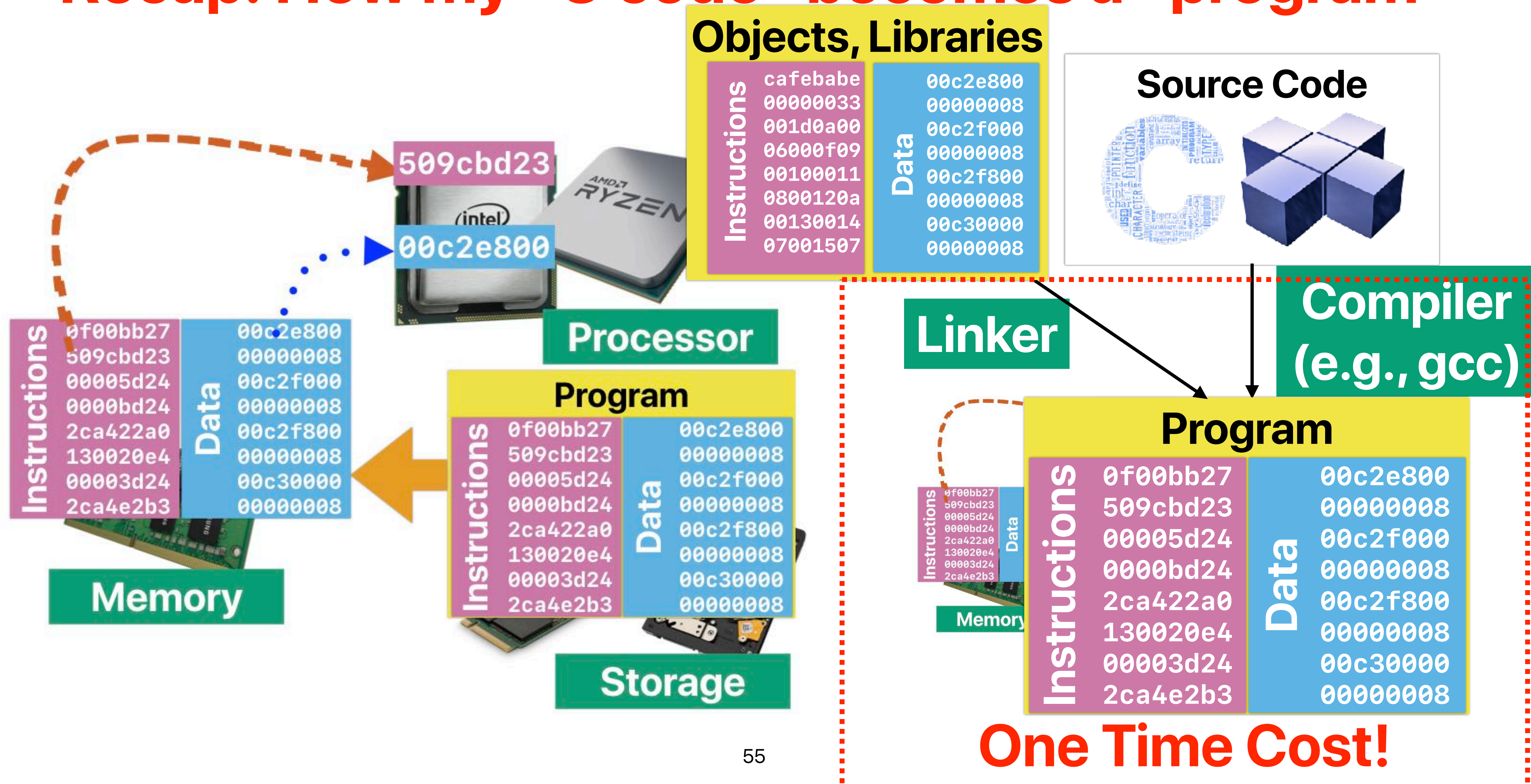
Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric —
Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions

What Affects Each Factor in Performance Equation

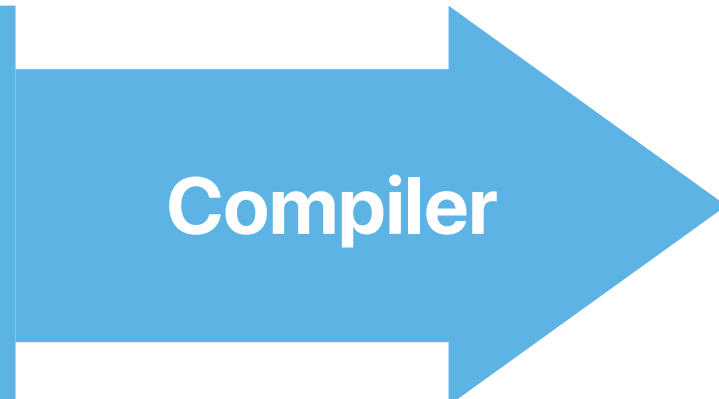
Programming languages & performance

Recap: How my "C code" becomes a "program"



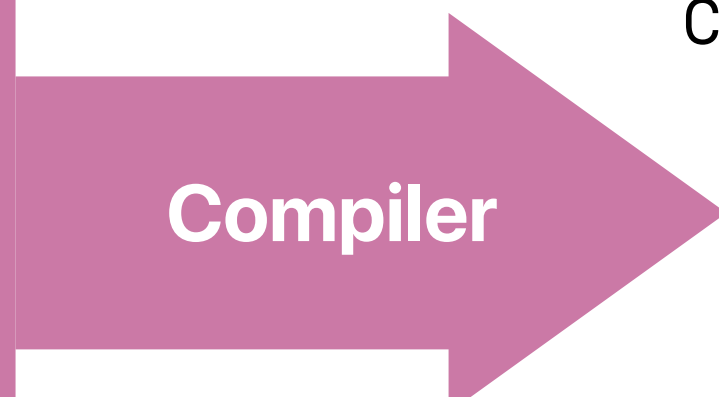
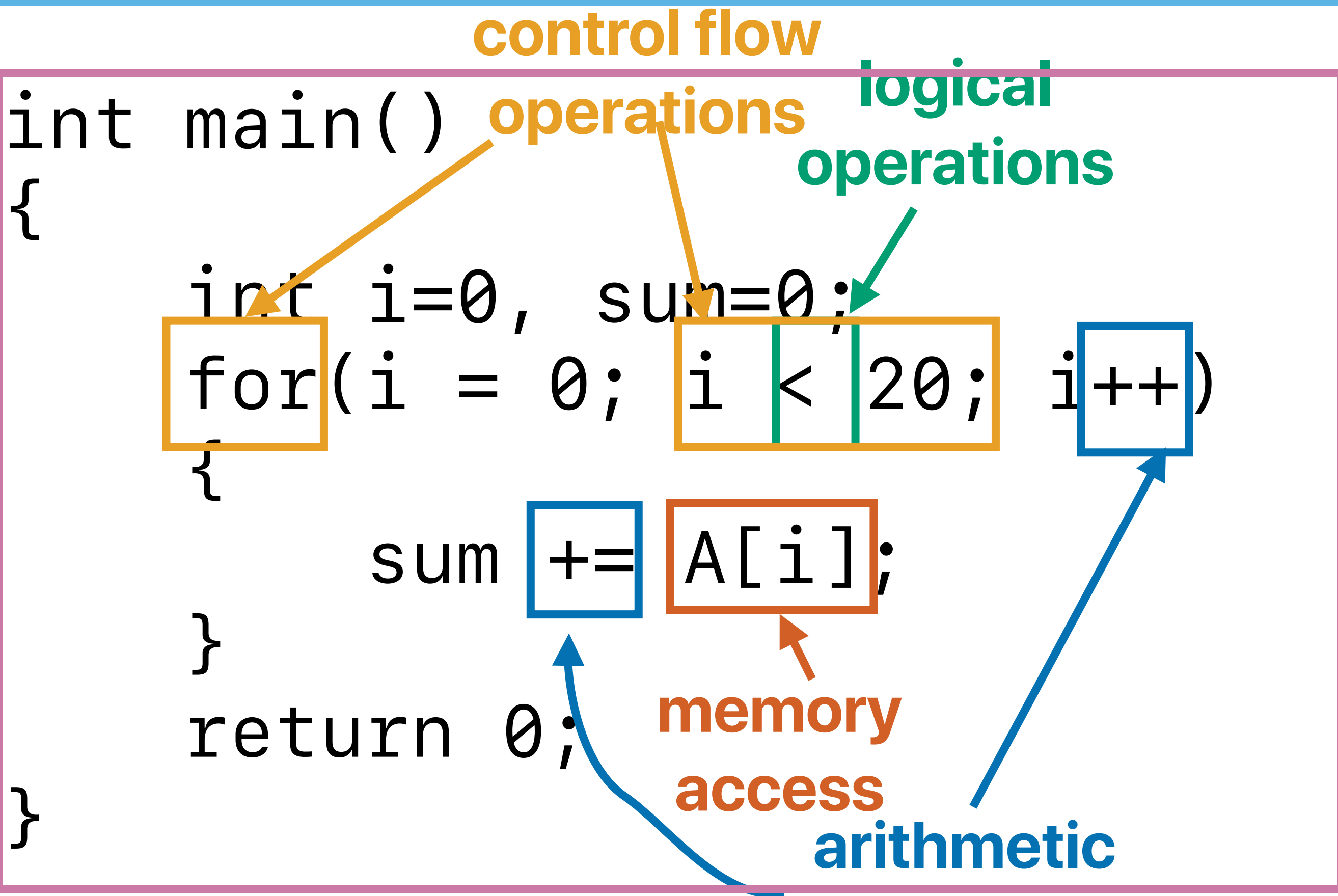
Start with this simple program in C

```
int A[] =
{1,2,3,4,5,6,7,8,9,10,1,2,3,4
,5,6,7,8,9,10};
```



Contents of section .data:

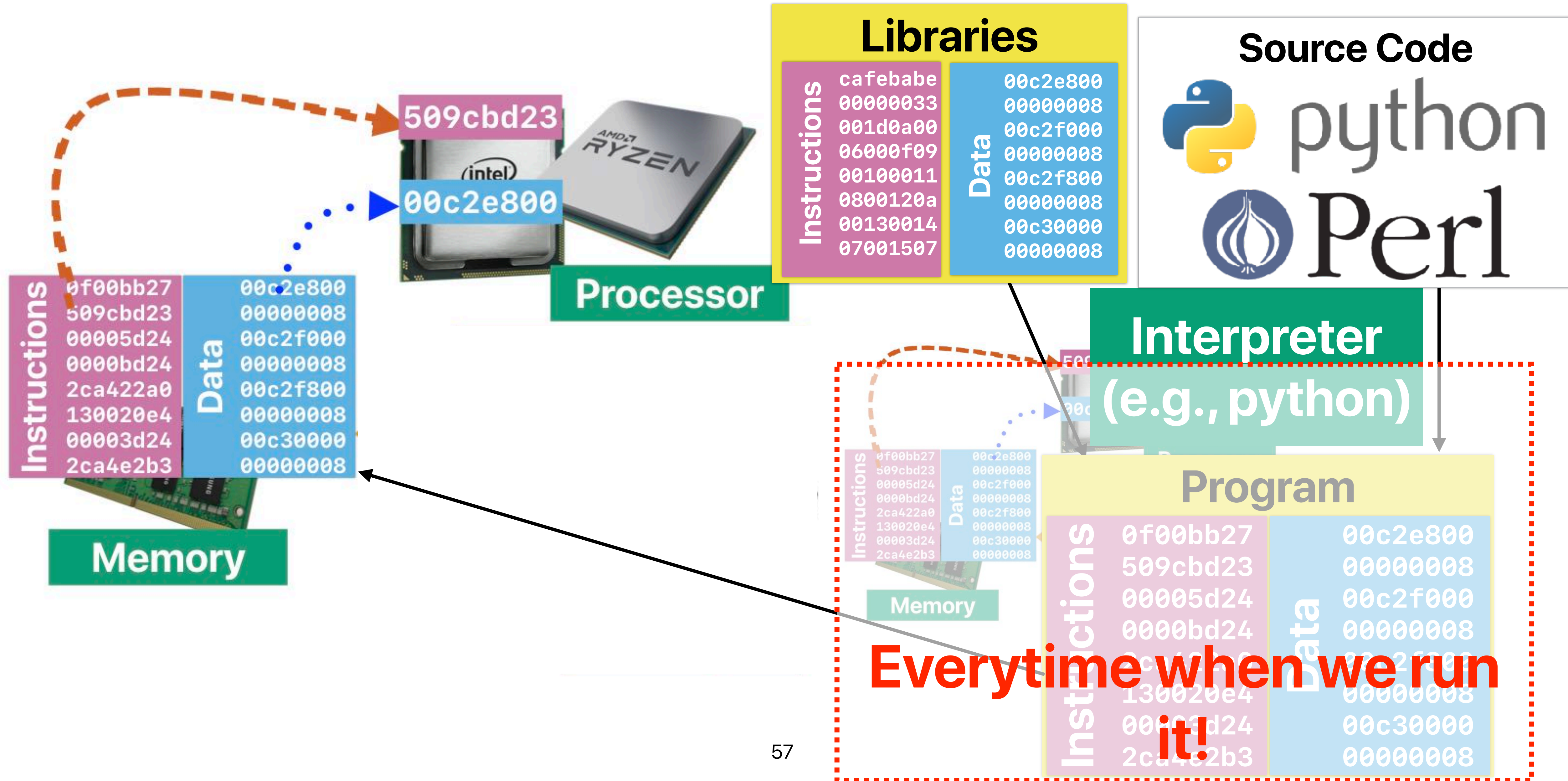
0000	01000000	02000000	03000000	04000000
0010	05000000	06000000	07000000	08000000
0020	09000000	0a000000	0b000000	0c000000
0030	0d000000	0e000000	0f000000	10000000
0040	11000000	12000000	13000000	14000000



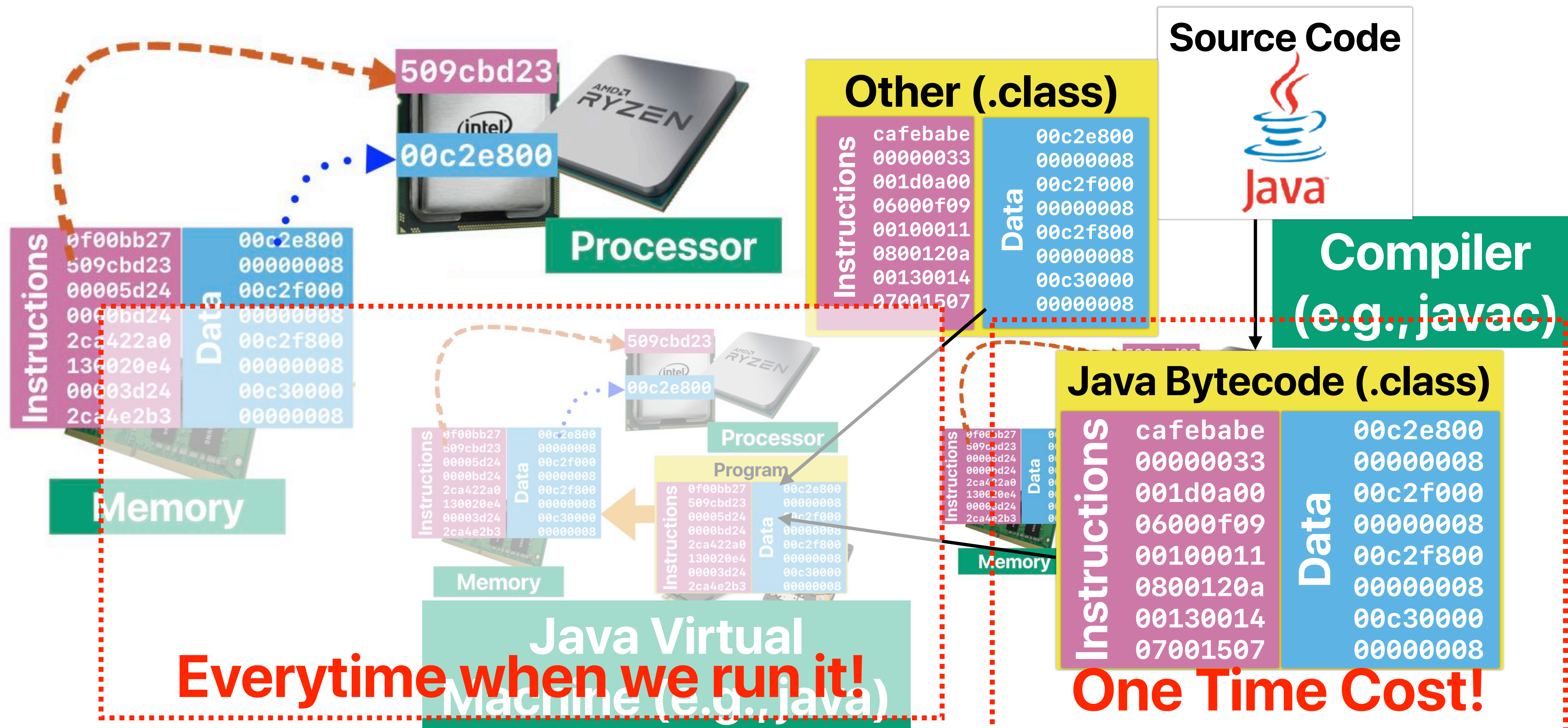
Contents of section .text:

0000	f30f1efa	554889e5	c745f800	000000c7
0010	45fc0000	0000c745	f8000000	00eb1e8b
0020	45f84898	488d1140	50000000	488d0500
0030	0000008b	04020145	fc8345f8	01837df8
0040	137edcb8	00000000	5dc3	

Recap: How my "Python code" becomes a "program"

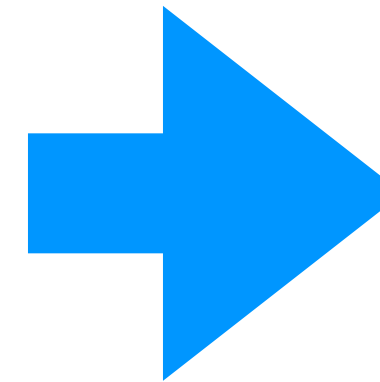


Recap: How my "Java code" becomes a "program"



What's in your java classes?

```
public static int fibonacci(int n) {  
    if(n == 0)  
        return 0;  
    else if(n == 1)  
        return 1;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

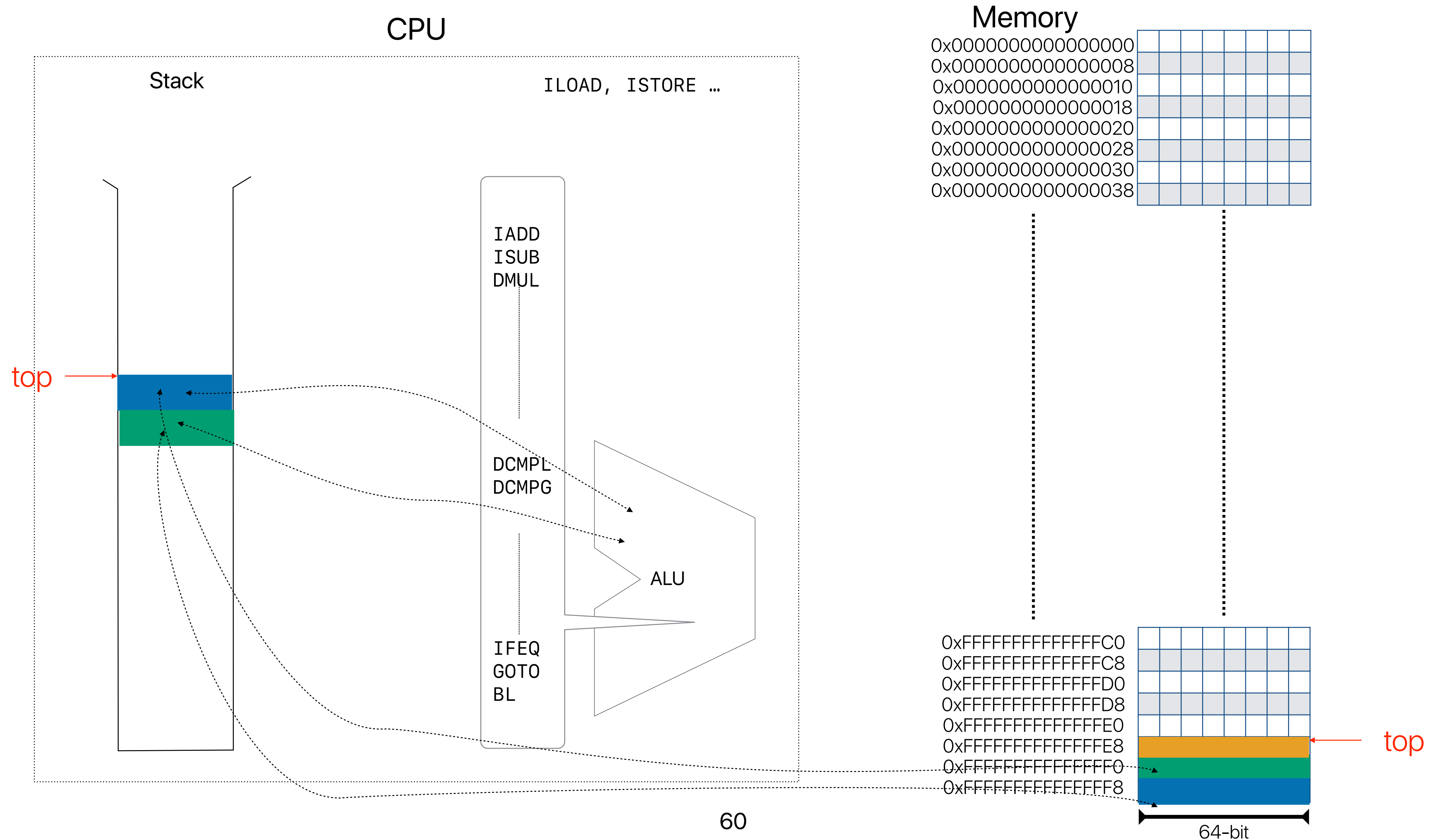


```
0: iload_0  
1: ifne  
4: iconst_0  
5: ireturn  
6: iload_0  
7: iconst_1  
8: if_icmpne  
11: iconst_1  
12: ireturn  
13: iload_0  
14: iconst_1  
15: isub  
16: invokestatic #2  
19: iload_0  
20: iconst_2  
21: isub  
22: invokestatic #2  
25: iadd  
26: ireturn
```

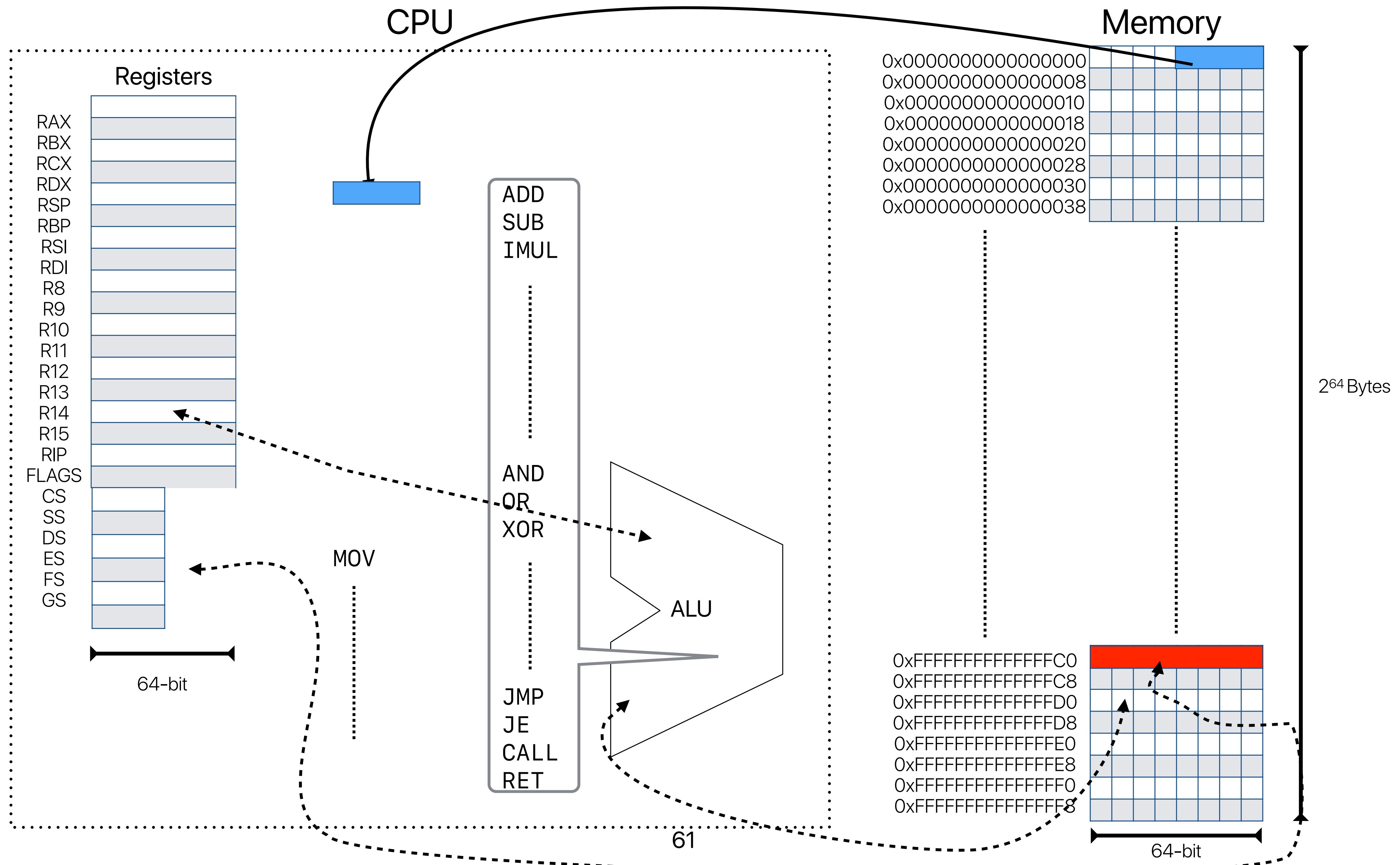
labels

**Most instructions doesn't
have an argument!**

"Abstracted" Java Architecture

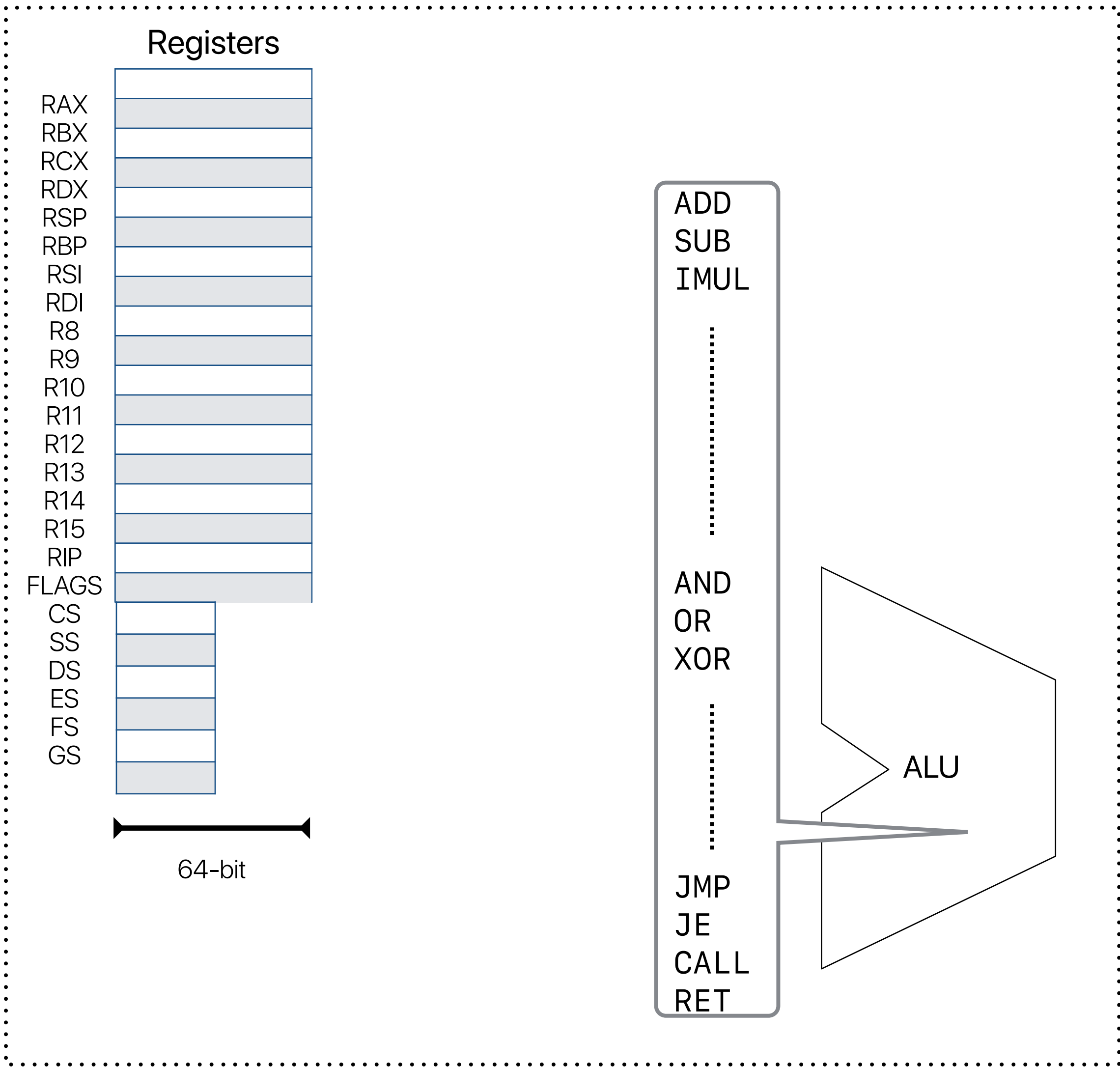


x86 ISA: the abstracted machine

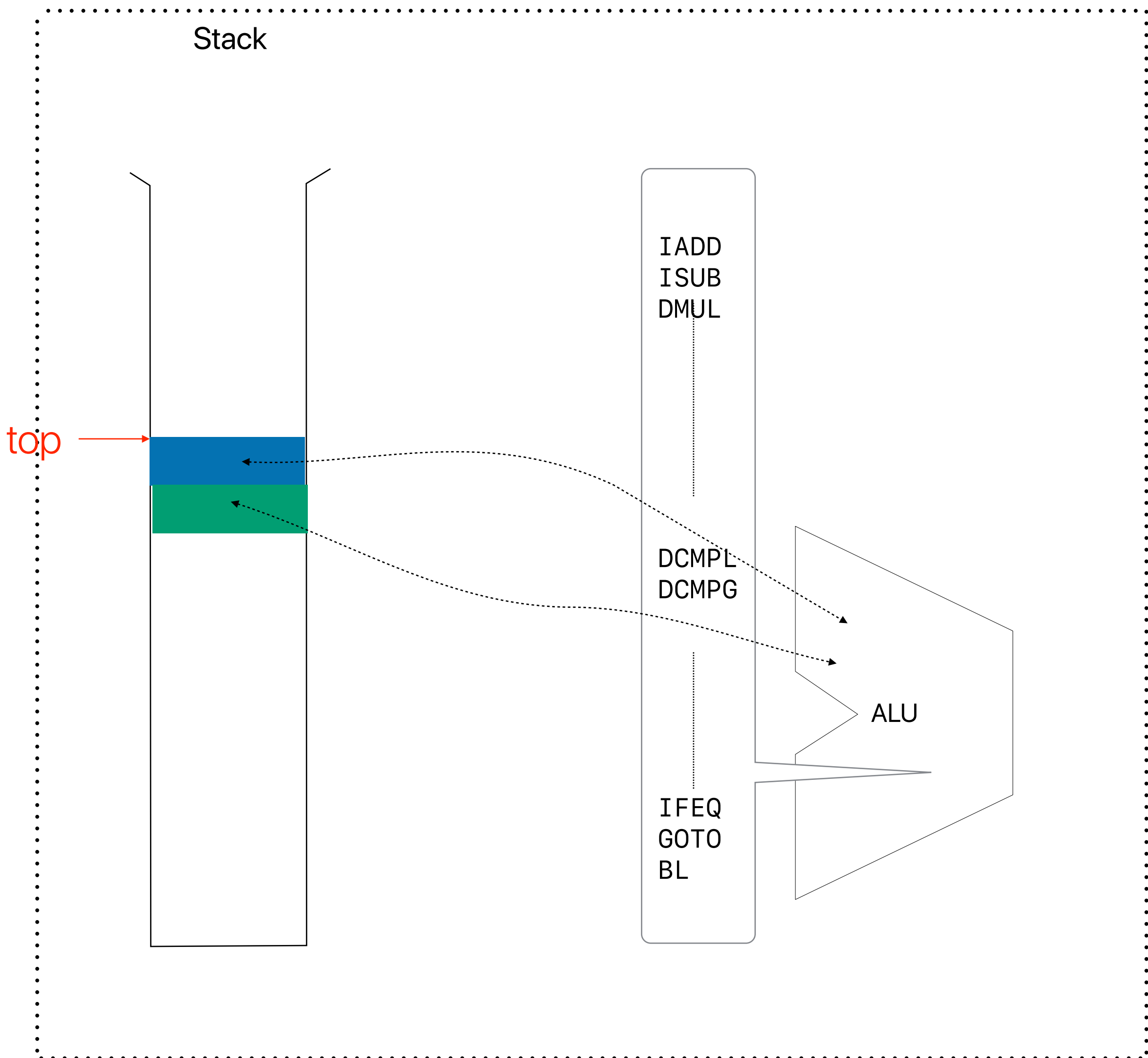


Mismatching real architectures vs JVM abstraction

x86 CPU



JVM's CPU abstraction



Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!

**What about programmer &
performance?**

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

Complexity

Instruction Count?

Clock Rate

CPI

Programmers can also set the cycle time

<https://software.intel.com/sites/default/files/comment/1716807/how-to-change-frequency-on-linux-pub.txt>

```
=====
Subject: setting CPU speed on running linux system
```

If the OS is Linux, you can manually control the CPU speed by reading and writing some virtual files in the "/proc"

1.) Is the system capable of software CPU speed control?

If the "directory" /sys/devices/system/cpu/cpu0/cpufreq exists, speed is controllable.

-- If it does not exist, you may need to go to the BIOS and turn on EIST and any other C and P state control and vi

2.) What speed is the box set to now?

Do the following:

```
$ cd /sys/devices/system/cpu
```

```
$ cat ./cpu0/cpufreq/cpuinfo_max_freq
```

```
3193000
```

```
$ cat ./cpu0/cpufreq/cpuinfo_min_freq
```

```
1596000
```

3.) What speeds can I set to?

Do

```
$ cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
```

It will list highest settable to lowest; example from my NHM "Smackover" DX58SO HEDT board, I see:

```
3193000 3192000 3059000 2926000 2793000 2660000 2527000 2394000 2261000 2128000 1995000 1862000 1729000 159600
```

You can choose from among those numbers to set the "high water" mark and "low water" mark for speed. If you set "h

4.) Show me how to set all to highest settable speed!

Use the following little sh/ksh/bash script:

```
$ cd /sys/devices/system/cpu # a virtual directory made visible by device drivers
```

```
$ newSpeedTop=`awk '{print $1}' ./cpu0/cpufreq/scaling_available_frequencies`
```

```
$ newSpeedLow=$newSpeedTop # make them the same in this example
```

```
$ for c in ./cpu[0-9]* ; do
```

```
> echo $newSpeedTop >${c}/cpufreq/scaling_max_freq
```

```
> echo $newSpeedLow >${c}/cpufreq/scaling_min_freq
```

```
> done
```

```
$
```

5.) How do I return to the default - i.e. allow machine to vary from highest to lowest?

Edit line # 3 of the script above, and re-run it. Change the line:

```
$ newSpeedLow=$newSpeedTop # make them the same in this example
```

To read

Check your assignment 1 regarding your power in cycle time!

Toggle Code Cells

6.3. Instruction Count

6.3.1. Running The Experiment Mutiple Times

6.3.2. Increasing the Size of array

6.3.3. Question 3 (Correctness)

6.4. Cycle Time

6.4.1. Question 4 (Completeness)

6.5. Cycles Per Instruction

6.5.1. Floating Point vs Integer Operations

6.5.2. Question 5 (Completeness)

6.5.3. Question 6 (Completeness)

6.5.4. The Compiler's Effect

6.5.5. Question 7 (Correctness)

6.5.6. Question 8 (Correctness)

6.5.7. Code Structure

6.5.8. Question 9 (Correctness)

6.5.9. Question 10 (Correctness)

7. Amdahl's Law

7.0.1. Question 11 (Correctness)

7.0.2. Question 12 (Challenging)

7.0.3. Question 13 (Challenging)

Python 3 (ipyker)

6.4. Cycle Time

Next, we'll take a look at how clock rate affects performance. Before we do, though, let's see what our options are for clock rate on our machine:

[26]: !cs203 run 'cpupower frequency-info -n'

analyzing CPU 0:
driver: intel_pstate
CPUs which run at the same hardware frequency: 0
CPUs which need to have their frequency coordinated by software: 0
maximum transition latency: Cannot determine or is not supported.
hardware limits: 800.000 MHz - 4.300000 GHz
available cpufreq governors: performance powersave
current policy: frequency should be within 800.000 MHz and 4.300000 GHz.
The governor "powersave" may decide which speed to use within this range.
current CPU frequency: Unable to call hardware
current CPU frequency: 800.000 MHz (asserted by call to kernel)
boost state support:
Supported: yes
Active: yes

As you can see, the processors in our target systems can run between 800MHz and 4300MHz.

Let's see how that affects things by plotting execution time as a function of clock speed (we are skipping 800MHz for the moment. We'll come back to it.). The readings for the current clock speed may vary from run to run. It just ends up at whatever the last experiment left it at.

Kick off the cell below to collect the data. While it's running answer this question:

[27]: !cs203 run './microbench.exe -o cycle_time.csv -M 800 1600 2000 3200 4000 4300 -f baseline_int -r 50'

Execution complete

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Programmers can control all three factors in the classic performance equation when composing the program

Compilers & performance

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Programmers can control all three factors in the classic performance equation when composing the program
- Compiler optimization does not always help
 - Compiler optimizes code based on some assumptions that may not be true on all computers
 - Programmers' can write code in a way facilitating optimizations!

What matters to the classical CPU performance equation?

$$Performance = \frac{1}{Execution\ Time}$$

$$Execution\ Time = \frac{Instructions}{Program} \times \frac{Cycles}{Instruction} \times \frac{Seconds}{Cycle}$$

$$ET = IC \times CPI \times CT$$

- IC (Instruction Count)
 - ISA, Compiler, algorithm, programming language, **programmer**
- CPI (Cycles Per Instruction)
 - Machine Implementation, microarchitecture, compiler, application, algorithm, programming language, **programmer**
- Cycle Time (Seconds Per Cycle)
 - Process Technology, microarchitecture, **programmer**

How about complexity?

How about “computational complexity”

- Algorithm complexity provides a good estimate on the performance if —
 - Every instruction takes exactly the same amount of time
 - Every operation takes exactly the same amount of instructions

These are unlikely to be true

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Programmers can control all three factors in the classic performance equation when composing the program
- Compiler optimization does not always help
 - Compiler optimizes code based on some assumptions that may not be true on all computers
 - Programmers' can write code in a way facilitating optimizations!
- Complexity does not provide good assessment on real machines due to the idealized assumptions

Quantitative Analysis of “Better”

Speedup

- The relative performance between two machines, X and Y. Y is n times faster than X

$$n = \frac{\textit{Execution Time}_X}{\textit{Execution Time}_Y}$$

- The speedup of Y over X

$$\textit{Speedup} = \frac{\textit{Execution Time}_X}{\textit{Execution Time}_Y}$$

Takeaways: What is and what affects “performance”?

- The only definition of Y is Speedup times faster than X —

$$Speedup = \frac{Execution\ Time_X}{Execution\ Time_Y}$$

COVER FEATURE

Amdahl's Law in the

Amdahl's Law — and It's

Implication in the Multicore Era

Augmenting Amdahl's law with a corollary for multicore hardware makes it relevant to future generations of chips with multiple processor cores. Obtaining optimal multicore performance will require further research in both extracting more parallelism and making sequential cores faster.

Mark D. Hill, University of Wisconsin-Madison

Michael R. Marty, Google

In IEEE Computer, vol. 41, no. 7

Amdahl's Law

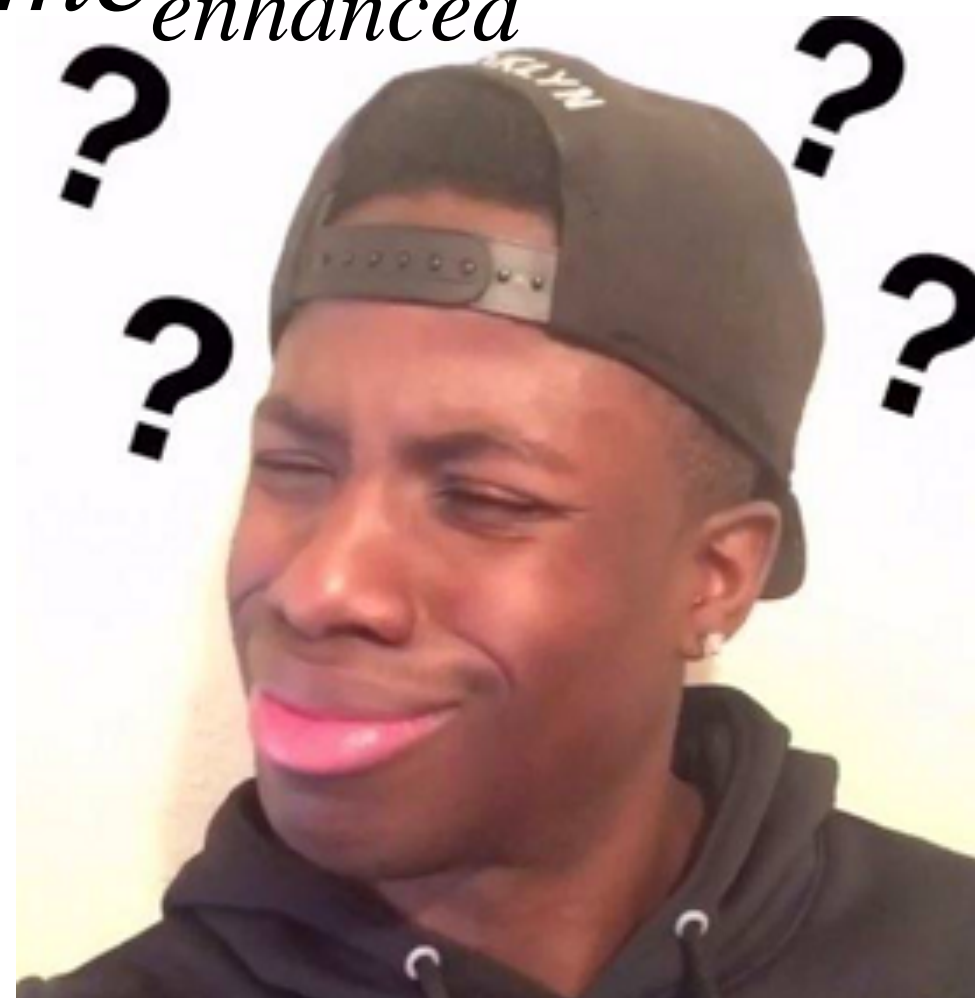


$$Speedup_{enhanced}(f, s) = \frac{1}{(1 - f) + \frac{f}{s}}$$

f — The fraction of time in the original program

s — The speedup we can achieve on f

$$Speedup_{enhanced} = \frac{Execution\ Time_{baseline}}{Execution\ Time_{enhanced}}$$



Amdahl's Law

$$Speedup_{enhanced}(f, s) = \frac{1}{(1 - f) + \frac{f}{s}}$$

Execution Time_{baseline} = 1

baseline



enhanced

Execution Time_{enhanced} = $(1 - f) + \frac{f}{s}$



$$Speedup_{enhanced} = \frac{Execution\ Time_{baseline}}{Execution\ Time_{enhanced}} = \frac{1}{(1 - f) + \frac{f}{s}}$$

FINAL FANTASY XV

POCKET EDITION **HD**



Amdahl's Law on Multiple Optimizations

- We can apply Amdahl's law for multiple optimizations
- These optimizations must be dis-joint!
 - If optimization #1 and optimization #2 are dis-joint:



$$Speedup_{enhanced}(f_{Opt1}, f_{Opt2}, s_{Opt1}, s_{Opt2}) = \frac{1}{(1 - f_{Opt1} - f_{Opt2}) + \frac{f_{Opt1}}{s_{Opt1}} + \frac{f_{Opt2}}{s_{Opt2}}}$$

- If optimization #1 and optimization #2 are not dis-joint:



$$Speedup_{enhanced}(f_{OnlyOpt1}, f_{OnlyOpt2}, f_{BothOpt1Opt2}, s_{OnlyOpt1}, s_{OnlyOpt2}, s_{BothOpt1Opt2}) = \frac{1}{(1 - f_{OnlyOpt1} - f_{OnlyOpt2} - f_{BothOpt1Opt2}) + \frac{f_{BothOpt1Opt2}}{s_{BothOpt1Opt2}} + \frac{f_{OnlyOpt1}}{s_{OnlyOpt1}} + \frac{f_{OnlyOpt2}}{s_{OnlyOpt2}}}$$

Amdahl's Law Corollary #1

- The maximum speedup is bounded by

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f) + \frac{f}{\infty}}$$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

Corollary #2 — make the common case fast!

- When f is small, optimizations will have little effect.
- Common == **most time consuming** not necessarily the most frequent
- The uncommon case doesn't make much difference
- The common case can change based on inputs, compiler options, optimizations you've applied, etc.

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$
 $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 1 — each optimization has an upper bound

- Corollary 2 — make the common case (the most time consuming case) fast!

$$\begin{aligned} Speedup_{max}(f_1, \infty) &= \frac{1}{(1-f_1)} \\ Speedup_{max}(f_2, \infty) &= \frac{1}{(1-f_2)} \\ Speedup_{max}(f_3, \infty) &= \frac{1}{(1-f_3)} \\ Speedup_{max}(f_4, \infty) &= \frac{1}{(1-f_4)} \end{aligned}$$

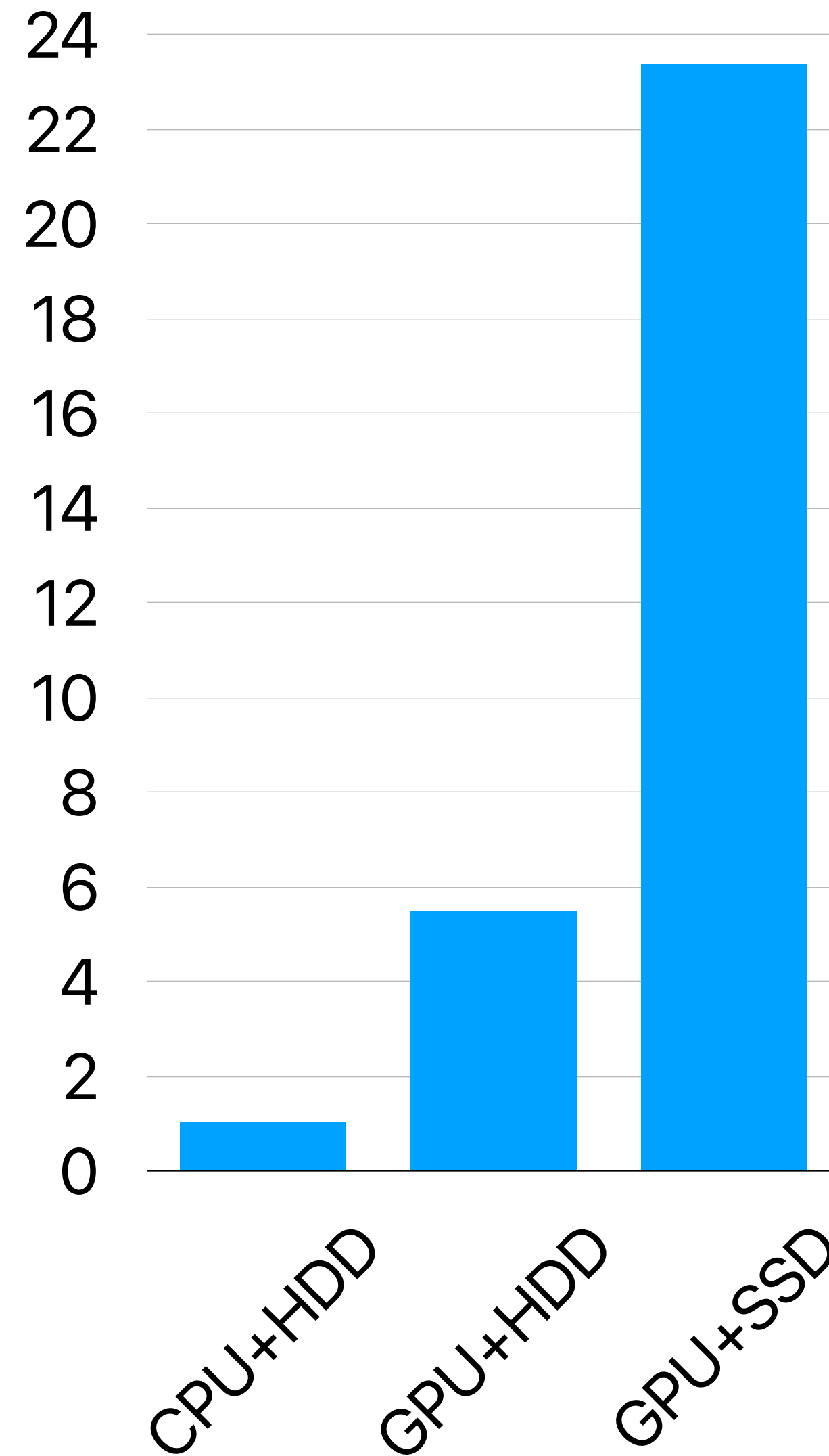
Identify the most time consuming part

- Use perf
 - perf record command
 - perf report —stdio
- Use prof
 - Compile your program with -pg flag
 - Run the program
 - It will generate a gmon.out
 - gprof your_program gmon.out > your_program.prof
 - It will give you the profiled result in your_program.prof
- Or insert timestamps

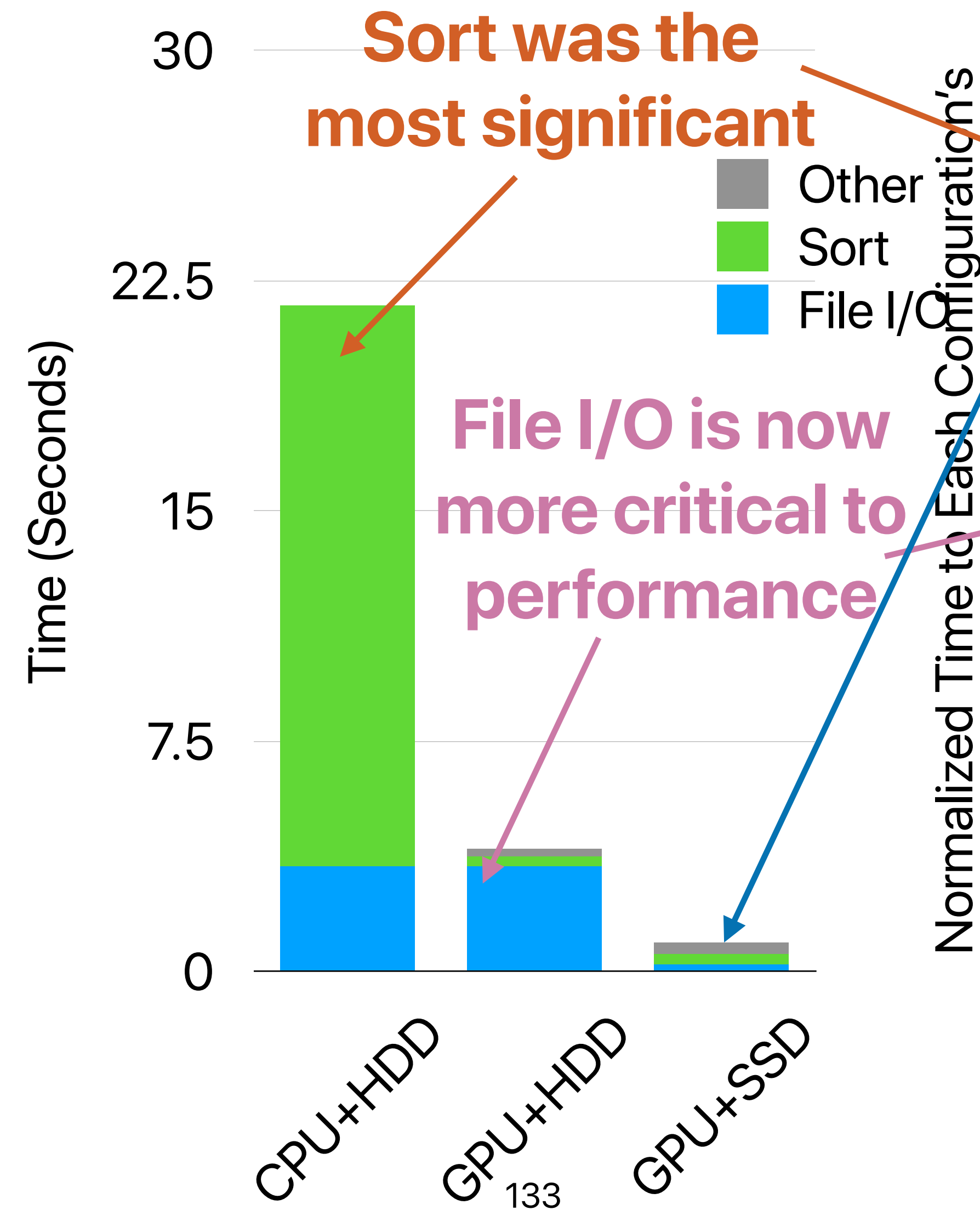
Demo — sort

Something else (e.g., data movement) matters more

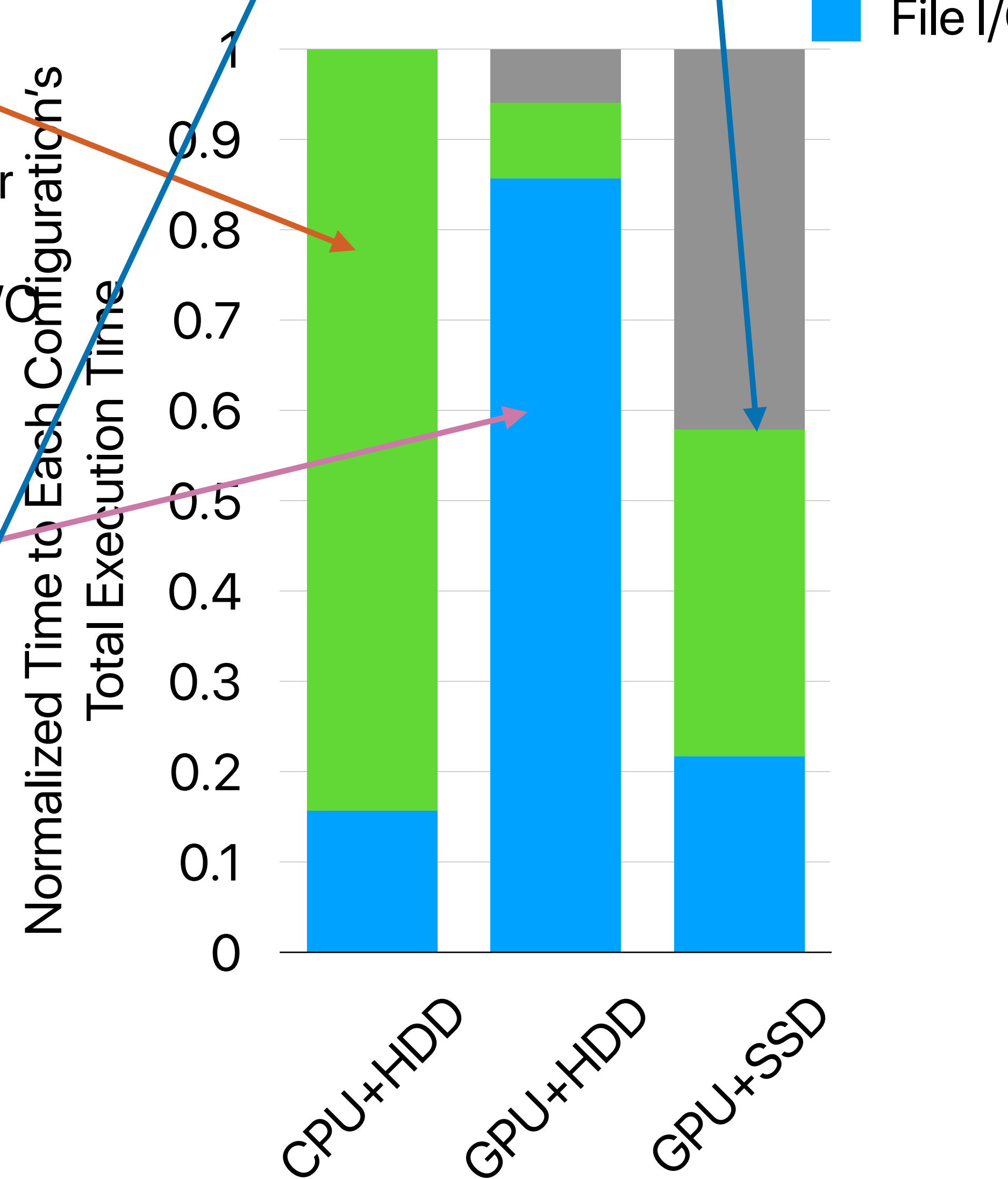
Speedup



Cumulative Execution Time



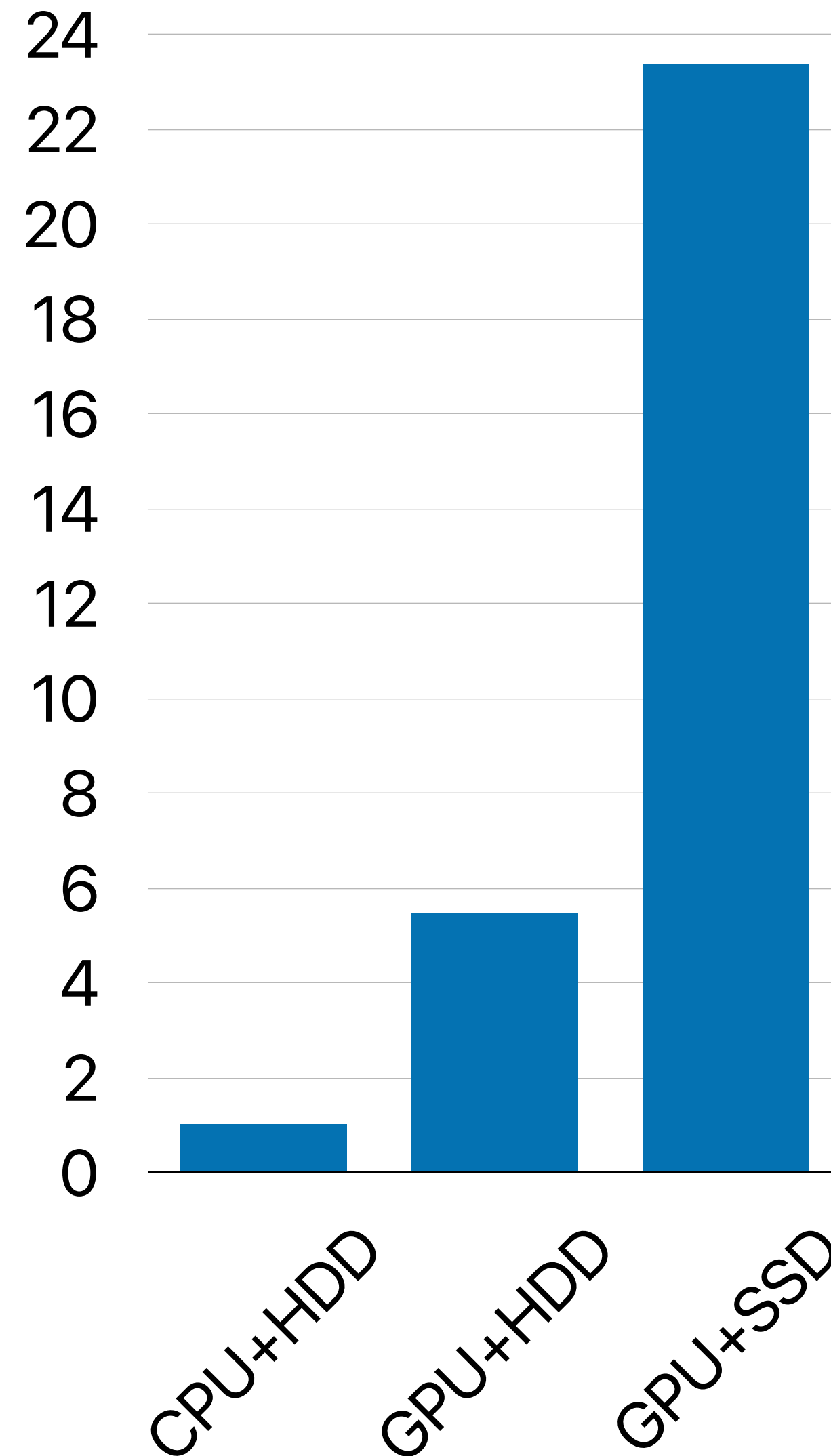
Execution Time Breakdown



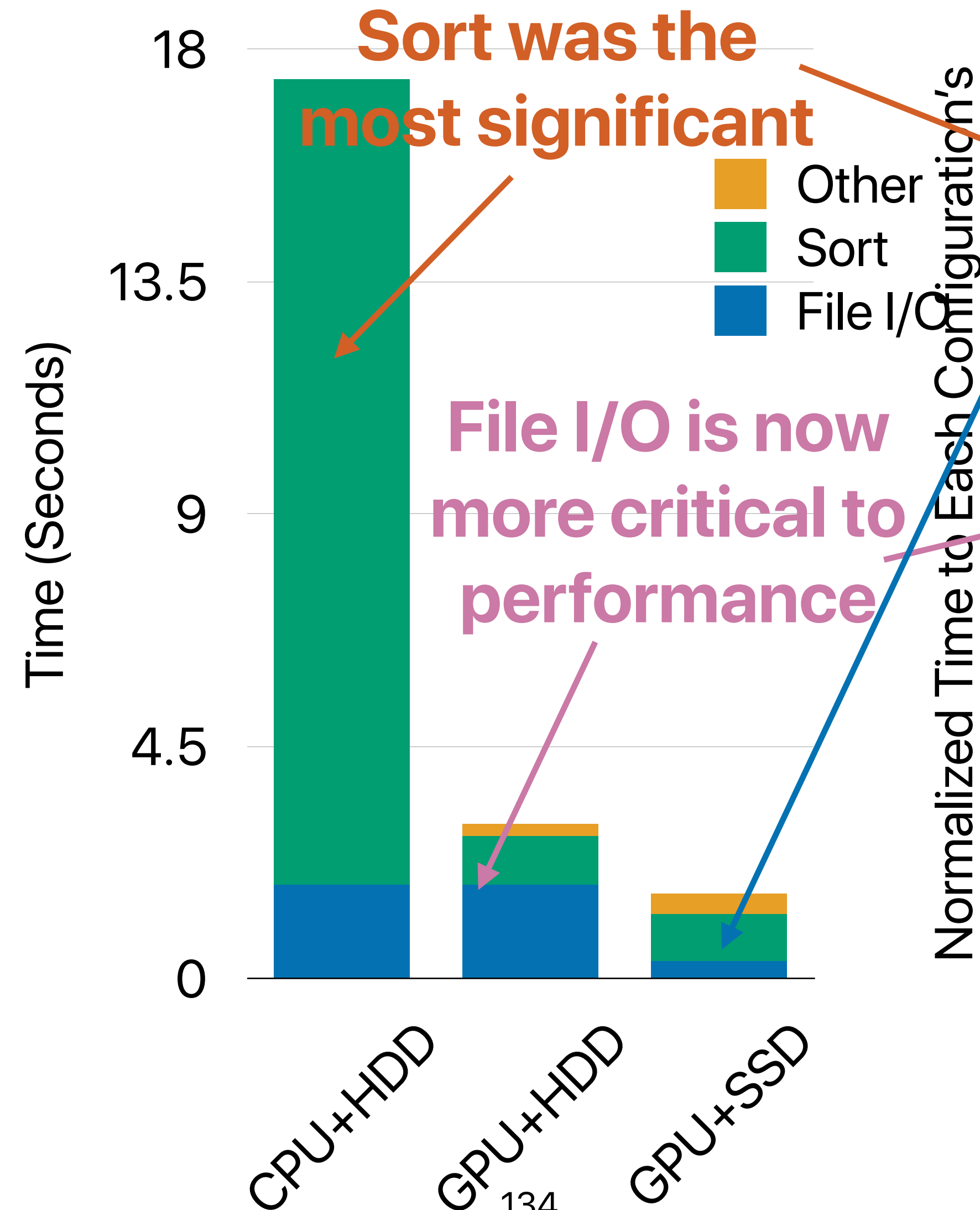
Demo — sort

Something else (e.g., data movement) matters more

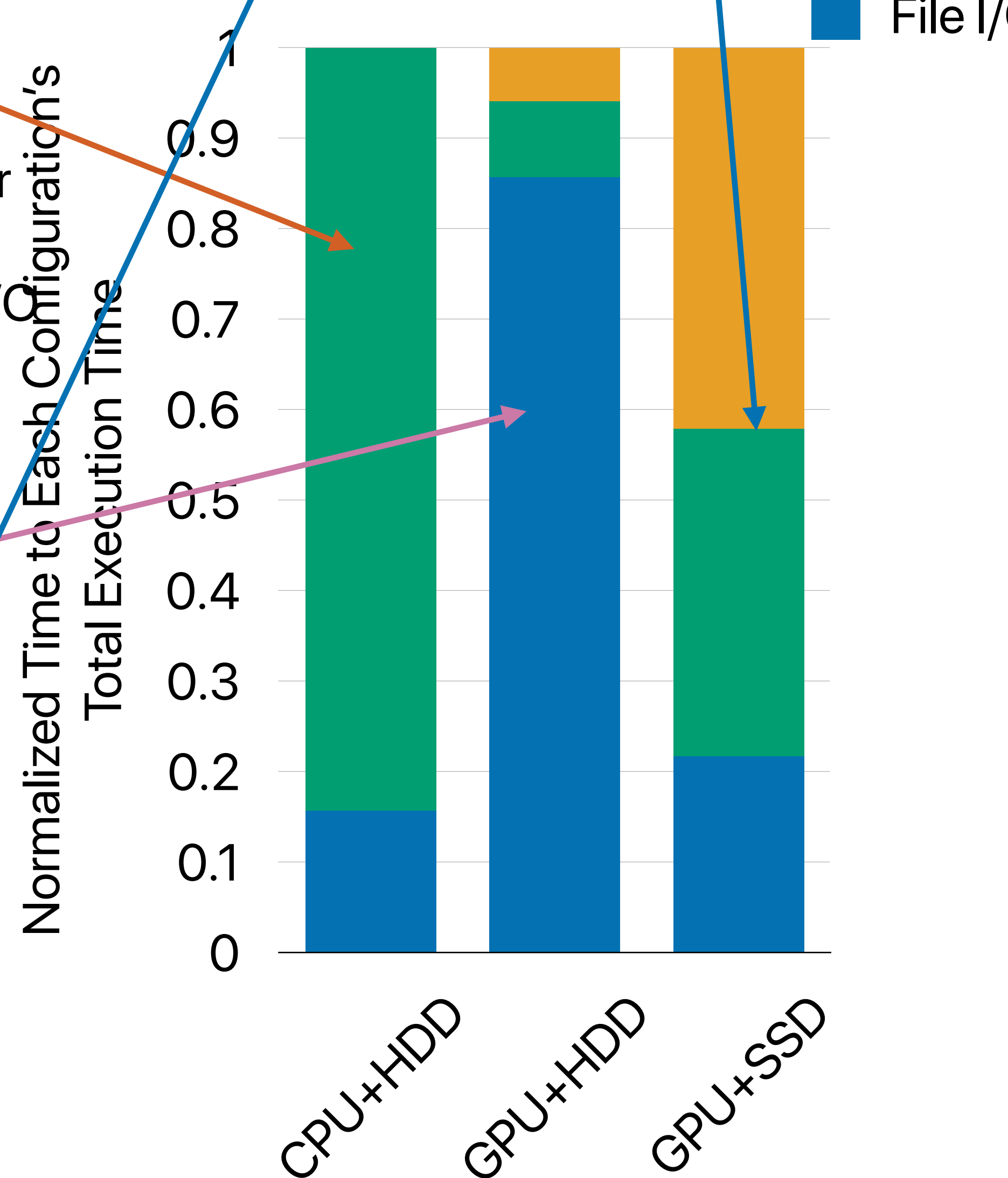
Speedup



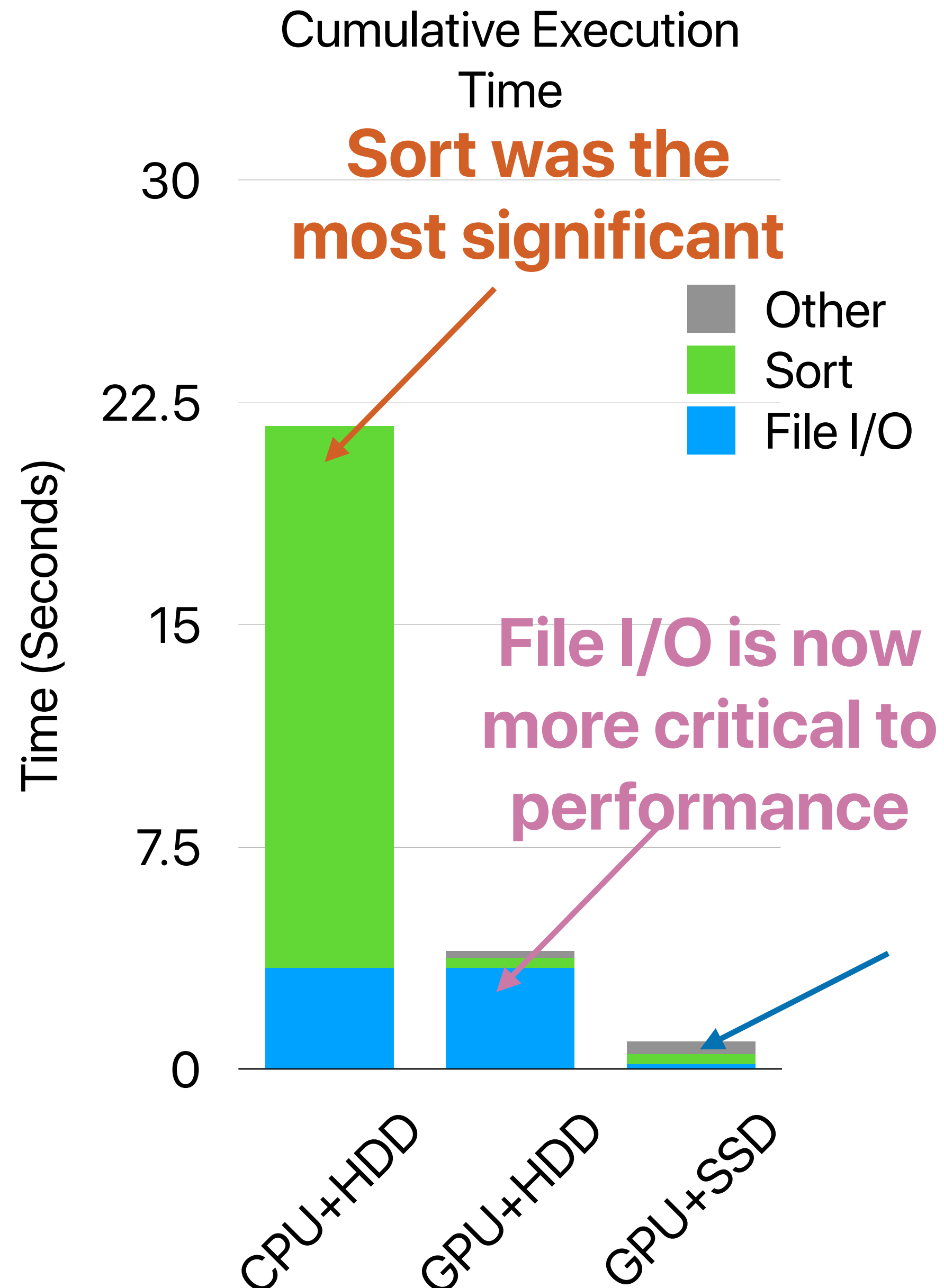
Cumulative Execution Time



Execution Time Breakdown



Corollary #3: optimization has a moving target



- With optimization, the common becomes uncommon.
- An uncommon case will (hopefully) become the new common case.
- Now you have a new target for optimization — You have to revisit "Amdahl's Law" every time you applied some optimization

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$
 $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 1 — each optimization has an upper bound

- Corollary 2 — make the common case (the most time consuming case) fast!

- Corollary 3 — Optimization has a moving target

$$\begin{aligned} Speedup_{max}(f_1, \infty) &= \frac{1}{(1-f_1)} \\ Speedup_{max}(f_2, \infty) &= \frac{1}{(1-f_2)} \\ Speedup_{max}(f_3, \infty) &= \frac{1}{(1-f_3)} \\ Speedup_{max}(f_4, \infty) &= \frac{1}{(1-f_4)} \end{aligned}$$

Amdahl's Law on Multicore Architectures

- Symmetric multicore processor with n cores (if we assume the processor performance scales perfectly)

$$Speedup_{parallel}(f_{parallelizable}, n) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{n}}$$

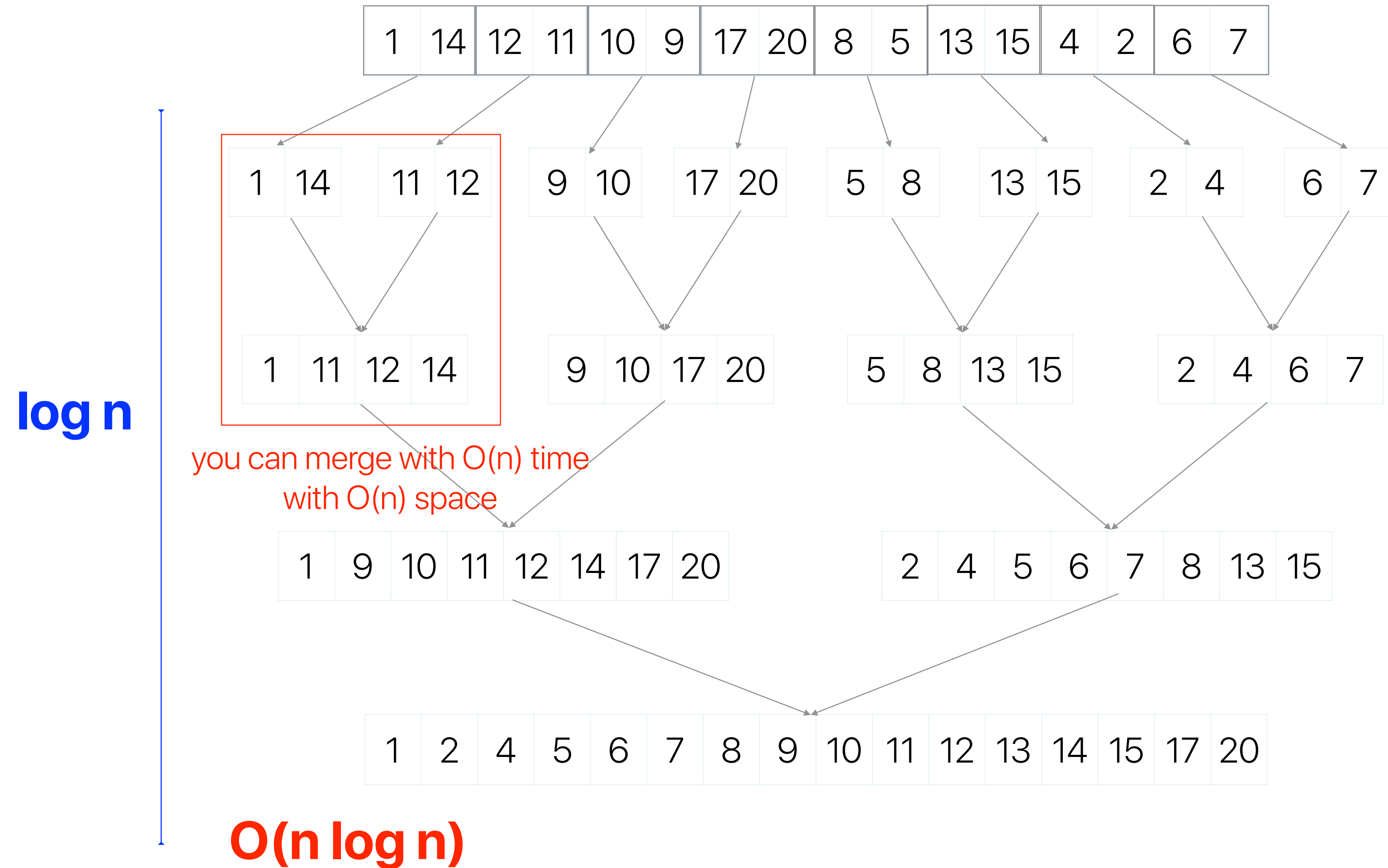
Demo — merge sort v.s. bitonic sort on GPUs

Merge Sort
 $O(n \log_2 n)$

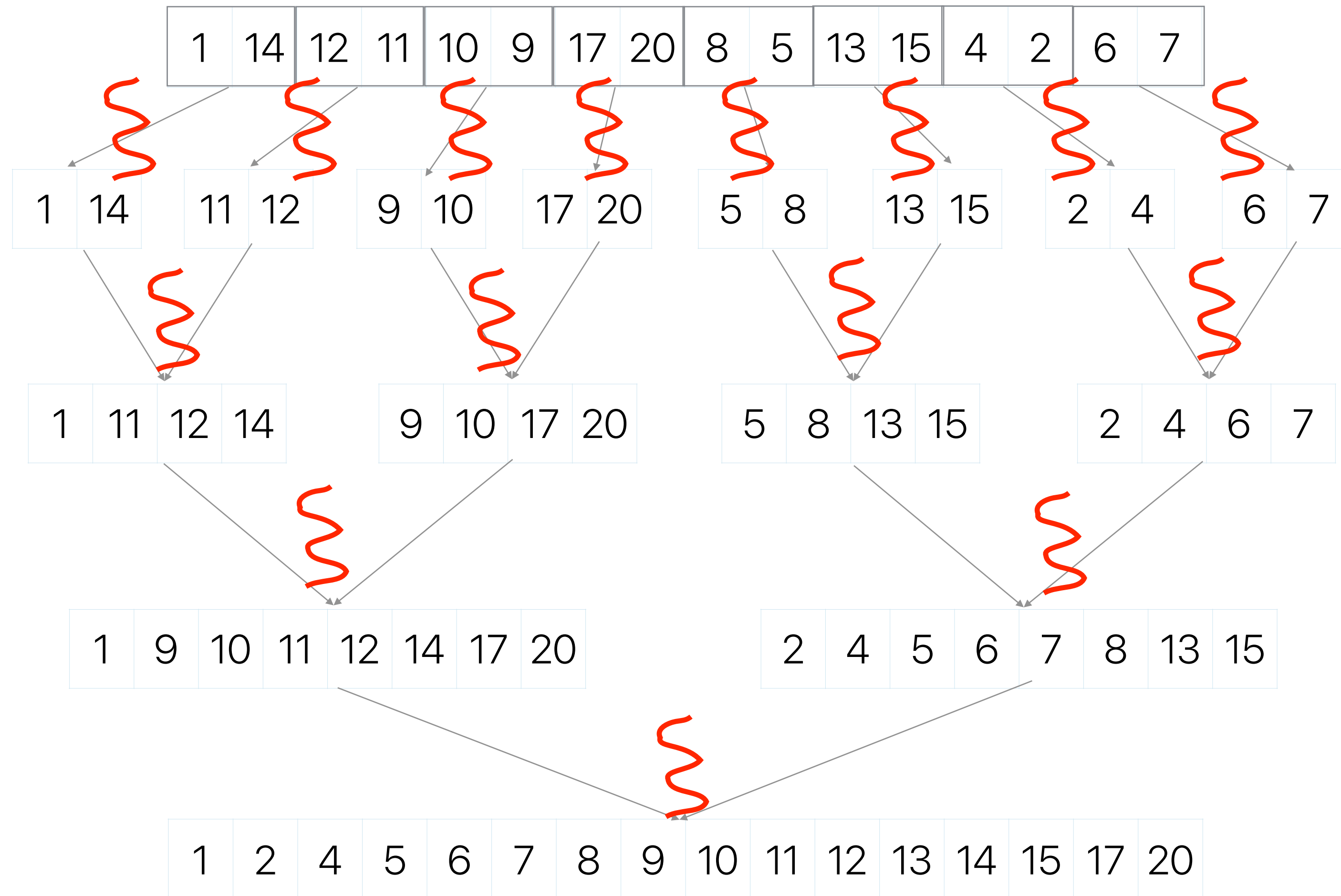
Bitonic Sort
 $O(n \log_2^2 n)$

```
void BitonicSort() {  
  
    int i,j,k;  
  
    for (k=2; k<=N; k=2*k) {  
        for (j=k>>1; j>0; j=j>>1) {  
            for (i=0; i<N; i++) {  
                int ij=i^j;  
                if ((ij)>i) {  
                    if ((i&k)==0 && a[i] > a[ij])  
                        exchange(i,ij);  
                    if ((i&k)!=0 && a[i] < a[ij])  
                        exchange(i,ij);  
                }  
            }  
        }  
    }  
}
```

Merge sort



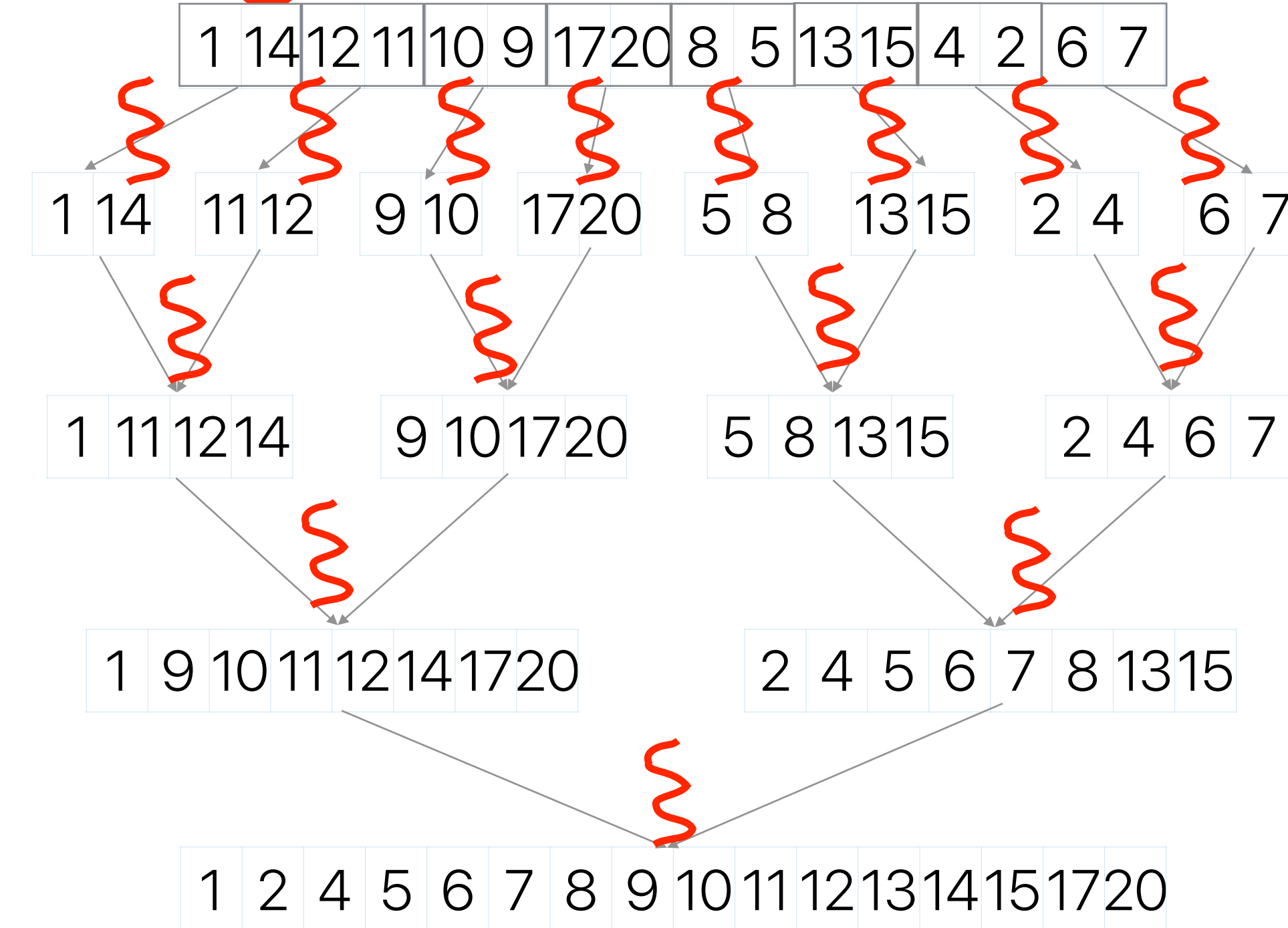
Parallel merge sort



What's the speedup of merge sort using Amdahl's Law

The degree of parallelism is $1, 2, 4, \dots, \frac{n}{2}$
at step $1, 2, 3, \dots, \log_2(n)$

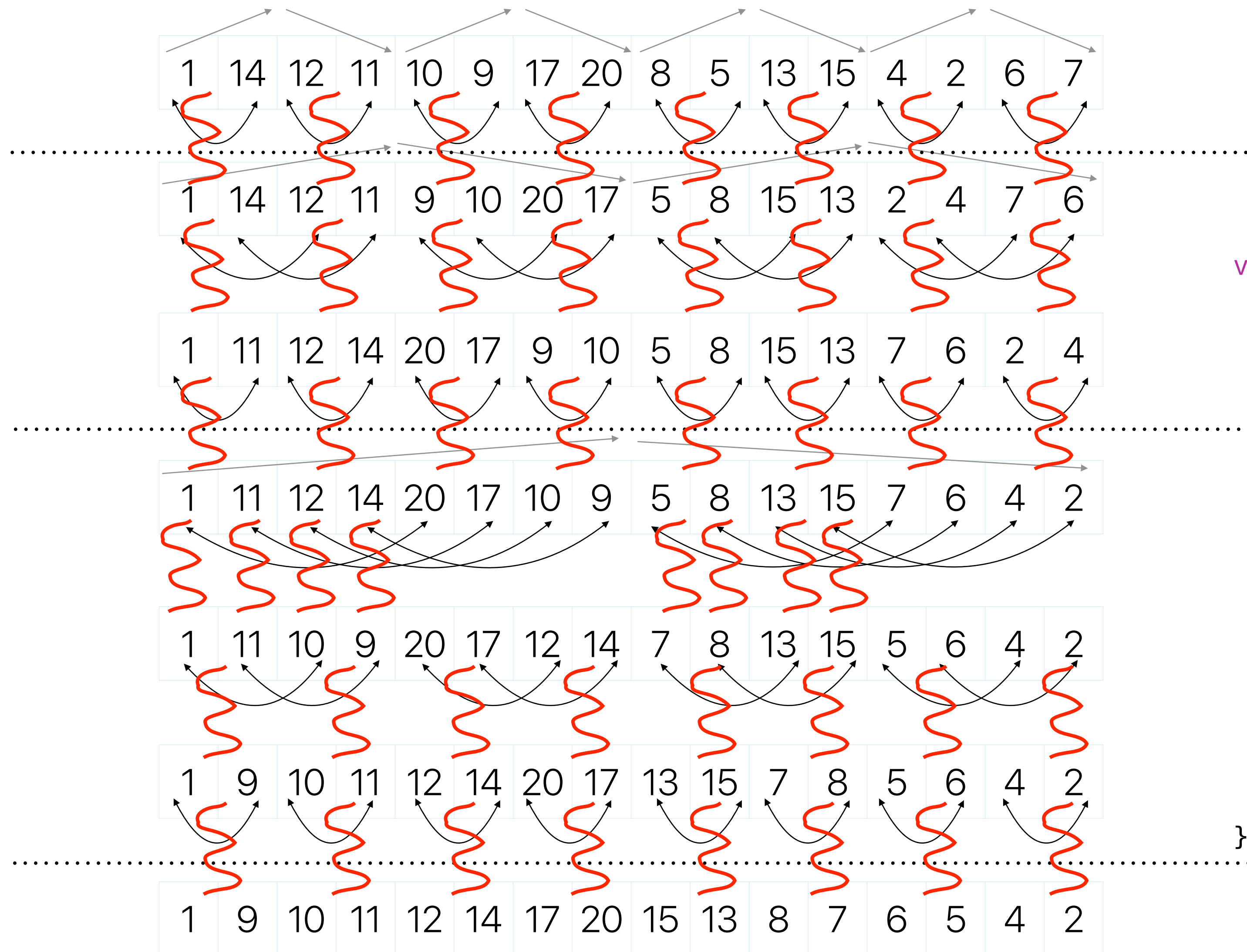
The ideal speedup of each step is $1, 2, 4, \dots, \frac{n}{2}$ or say
 $1, 2, 4, \dots, 2^{\log_2(n)-1}$ if we have **unlimited** parallelism
if we assume equal amount of time in each step in the baseline,
each step is going to take $\frac{1}{\lg(n)}$ portion of time in the baseline, and
the first $\frac{1}{\lg(n)}$ is not parallelizable (i.e., $(1 - x) = \frac{1}{\lg(n)}$)



So the Amdahl's Law's evaluation will become

$$\begin{aligned} \frac{1}{\frac{1}{\lg(n)} + \frac{1}{\lg(n) \times 2} + \frac{1}{\lg(n) \times 4} + \dots + \frac{1}{\lg(n) \times 2^{\lg(n)-1}}} &= \frac{1}{\frac{1}{\lg(n)}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{2^{\lg(n)-1}})} \\ &= \frac{1}{\frac{1}{\lg(n)}(1 + 1 - \frac{1}{2^{\lg(n)-1}})} = \frac{1}{\frac{2}{\lg(n)}} = \frac{\lg(n)}{2} \end{aligned}$$

Bitonic sort



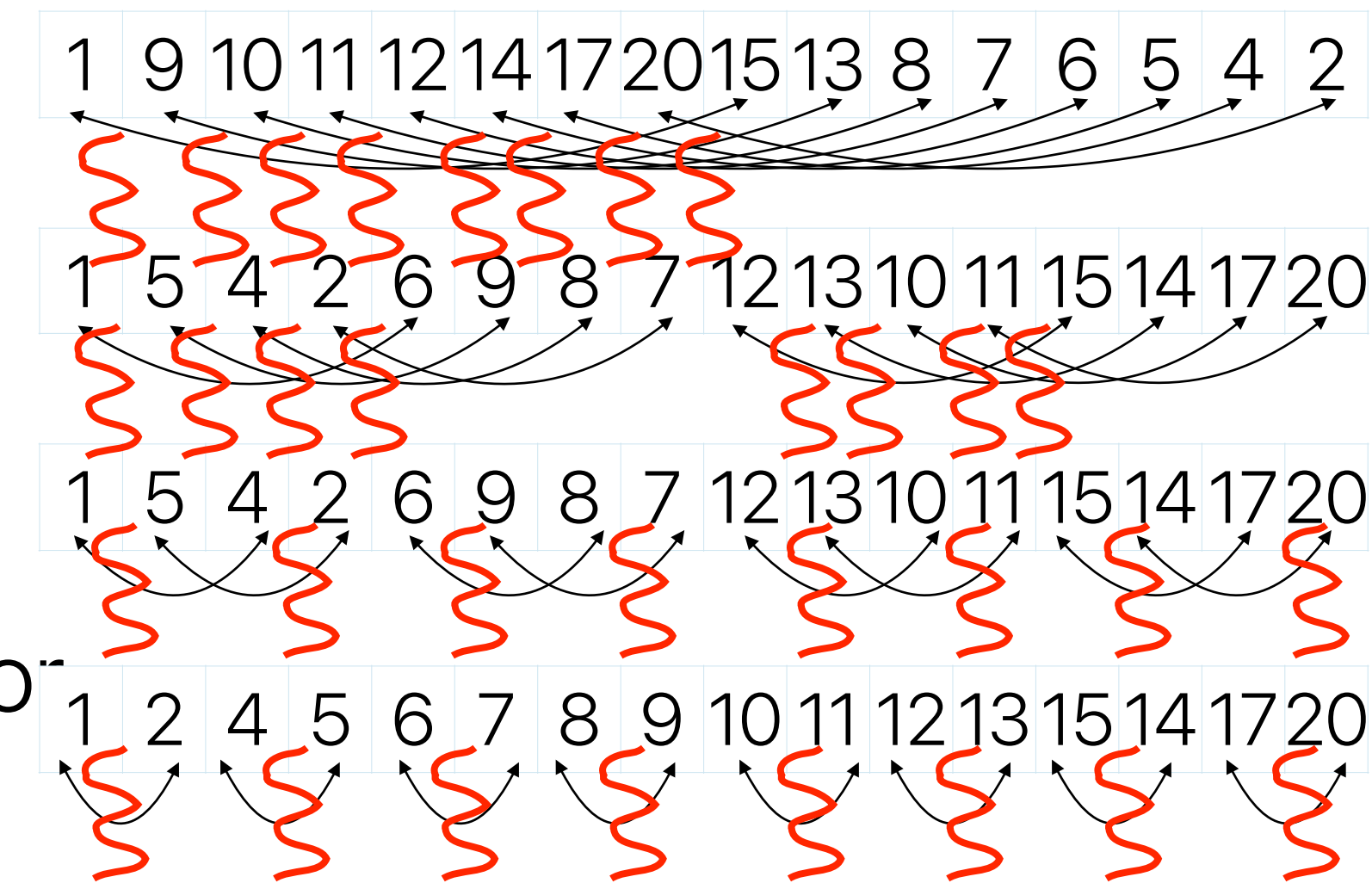
```
void BitonicSort() {
    int i,j,k;

    for (k=2; k<=N; k=2*k) {
        for (j=k>>1; j>0; j=j>>1) {
            for (i=0; i<N; i++) {
                int ij=i^j;
                if ((ij)>i) {
                    if ((i&k)==0 && a[i] > a[ij])
                        exchange(i,ij);
                    if ((i&k)!=0 && a[i] < a[ij])
                        exchange(i,ij);
                }
            }
        }
    }
}
```

What's the speedup of bitonic sort using Amdahl's Law

The degree of parallelism is always $\frac{n}{2}$
at step 1, 2, 3, ..., $\log_2^2(n)$

The ideal speedup of each step is $\frac{n}{2}$ if we have **unlimited** parallelism & for each step of bitonic sort. However, bitonic sort will have $\lg(n)$ x more steps than merge sort.



If the baseline is merge sort — the speedup of using bitonic sort is

$$\frac{\frac{1}{1 \times \lg(n)}}{\frac{n}{2}} = \frac{n}{2 \lg(n)} > \frac{\lg(n)}{2}$$

What if we have only p processors?

For **bitonic sort**, The degree of parallelism is always $\frac{n}{2}$ at step 1, 2, 3, ..., $\log_2^2(n)$, but we have only p processors...

The theoretical speedup of each step is $\max(\frac{n}{2}, p) = p$ since n is very likely to be larger than **p** & assume equal amount of time in each step in the baseline

So the Amdahl's Law's evaluation will become $\frac{1}{\frac{p}{\lg(n)}} = \frac{p}{\lg(n)} = \frac{1024}{30} = 34.1333333$

What about merge sort? $= \frac{1}{\frac{1}{\lg(n)}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{p})}$

What if $p=1024, n=1M=2^{30}$? $= \frac{1}{\frac{1}{30}(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{1024} + 20 \times \frac{1}{30})} = 14.862119$

Corollary #4

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{\infty}}$$

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable})}$$

- If we can build a processor with unlimited parallelism
 - The algorithm complexity becomes less important as long as the algorithm can utilize all parallelism
 - That's why bitonic sort or MapReduce works!
- **The future trend of software/application design is seeking for more parallelism rather than lower the computational complexity**

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$
- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$
 - Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$
 - Corollary 2 — make the common case (the most time consuming case) fast!
 $Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$
 $Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$
 $Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$
 $Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$
 - Corollary 3 — Optimization has a moving target
 - Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures
 $Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$

**Is it the end of computational
complexity?**

Corollary #5

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable}) + \frac{f_{parallelizable}}{\infty}}$$

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1 - f_{parallelizable})}$$

- Single-core performance still matters
 - It will eventually dominate the performance
 - If we cannot improve single-core performance further, finding more "parallelizable" parts is more important
 - Algorithm complexity still gives some "insights" regarding the growth of execution time in the same algorithm, though still not accurate

Takeaways: find the right thing to do

- Definition of "Speedup of Y over X" or say Y is n times faster than X: $speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

$$Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$$

- Corollary 1 — each optimization has an upper bound

- Corollary 2 — make the common case (the most time consuming case) fast!

$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$

- Corollary 3 — Optimization has a moving target

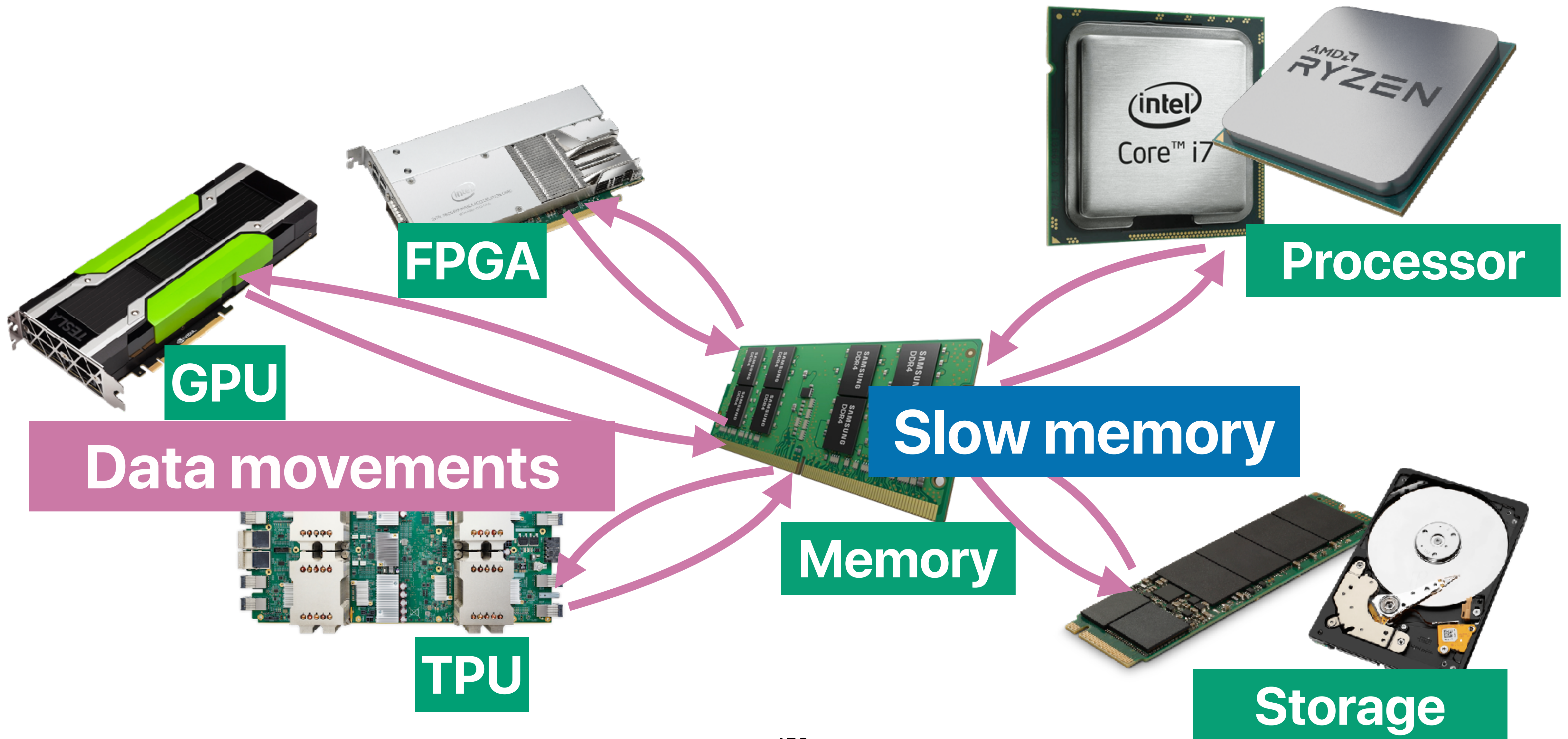
- Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 5 — Single-core performance still matters

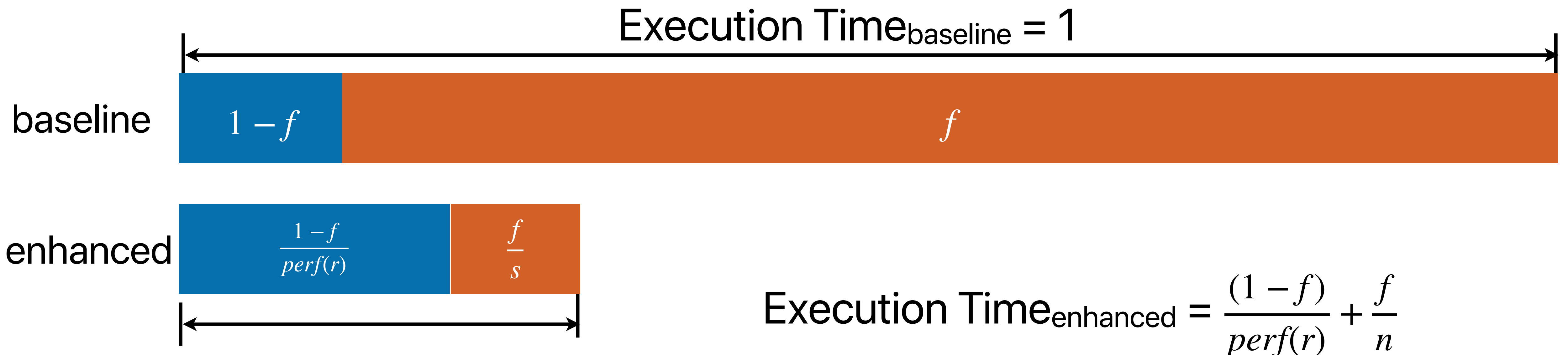
$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

Most cases, parallelism is not "tax-free"



What if parallelism is not "tax-free"?

- Parallelization Overhead
 - Preparing/exchanging/synchronizing data
 - Additional function calls/control overhead
- The " $1 - f$ " will potentially slowdown by a factor of $perf(r)$



Amdahl's Law considering overhead

$$Speedup_{enhanced}(f, s, r) = \frac{1}{\frac{(1-f)}{perf(r)} + \frac{f}{s}}$$

- r is some other parameter that affects the overhead
 - input size?
 - degree of parallelism?
- $perf(r)$ should be ≤ 1 (i.e., slowdown)
- The overhead may scale differently than the original problem — that's why we introduce " $perf()$ " function

Corollary #6: Don't hurt non-common part too much

- If the program spend 90% in A, 10% in B. Assume that an optimization can accelerate A by 9x, by hurts B by 10x (i.e., a speedup of $\frac{1}{10}$)...

$$\textit{Speedup} = \frac{1}{\frac{(1-f)}{\textit{perf}(r)} + \frac{f}{s}} = \frac{1}{\frac{(1-0.9)}{\frac{1}{10}} + \frac{0.9}{9}} = 0.91 \times$$

Takeaways: find the right thing to do!

- Definition of "Speedup of Y over X" or say Y is n times faster than X:

$$speedup_{Y_over_X} = n = \frac{Execution\ Time_X}{Execution\ Time_Y}$$

- Amdahl's Law — $Speedup_{enhanced}(f, s) = \frac{1}{(1-f) + \frac{f}{s}}$

- Corollary 1 — each optimization has an upper bound $Speedup_{max}(f, \infty) = \frac{1}{(1-f)}$

- Corollary 2 — make the common case (the most time consuming case) fast!

$$Speedup_{max}(f_1, \infty) = \frac{1}{(1-f_1)}$$

$$Speedup_{max}(f_2, \infty) = \frac{1}{(1-f_2)}$$

$$Speedup_{max}(f_3, \infty) = \frac{1}{(1-f_3)}$$

$$Speedup_{max}(f_4, \infty) = \frac{1}{(1-f_4)}$$

- Corollary 3 — Optimization has a moving target

- Corollary 4 — Exploiting more parallelism from a program is the key to performance gain in modern architectures

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 5 — Single-core performance still matters

$$Speedup_{parallel}(f_{parallelizable}, \infty) = \frac{1}{(1-f_{parallelizable})}$$

- Corollary 6 — Don't hurt the non-common case too much

$$Speedup_{enhanced}(f, s, r) = \frac{1}{(1-f) + perf(r) + \frac{f}{s}}$$

Takeaways: What does “perfect” mean?

- Latency is the most fundamental performance metric
- Classic CPU performance equation —Instruction count (IC), cycles per instruction (CPI), cycle time (CT) define the latency of execution on CPUs
- Performance metrics without considering all three factors in the classic performance equation can mislead — anything throughput typically miss one of them
- CO₂e, energy efficiency and power become increasingly important — but latency/execution time still matters as

$$Energy = Power \times Execution_Time$$